



Instituto Politécnico de Tomar



Escola Superior de Tecnologia de Tomar

Final Project Report for the Cloud Computing and Virtualization

António Gonçalves

Beatriz Moreira

2025/2026

Abstract

Business continuity is a fundamental requirement for modern information systems, since organizations must keep critical services available even when failures, maintenance operations, or unexpected outages occur. IBM defines business continuity as the ability to withstand outages and continue operating important services according to predefined service-level agreements [1]. In this context, high availability (HA) and disaster recovery (DR) are complementary strategies: disaster recovery focuses on restoring services after a major disruption, while high availability aims to minimize or eliminate interruption through redundancy, automated failover, and continuous service operation.

This report analyzes the evolution of “Project O” from a single-node monolithic Vagrant-based application into a decoupled, multi-tier high-availability architecture. The original system concentrates the web server, PHP application runtime, PostgreSQL database, local file storage, sessions, and WebSocket service in one virtual machine, creating several limitations: single points of failure, resource contention, local state entrapment, fragile WebSocket behavior, and limited scalability.

To address these limitations, the proposed solution progressively decouples the main architectural layers. The infrastructure introduces redundant load balancers with Nginx and Keepalived, database high availability with Patroni and Consul, connection pooling with PgBouncer, centralized and highly available sessions with Redis Sentinel, cross-instance real-time messaging with RatchetPHP and Redis Pub/Sub, object storage with MinIO, and observability through Netdata, Loki, and Grafana. Each phase was validated through inspection, failure testing, and load testing with Vegeta, allowing the group to evaluate both resilience and performance impacts before moving to the next stage.

The results demonstrate that high availability is not achieved by a single technology, but by combining services, software, infrastructure, and operational procedures into a coherent architecture. The project started with an implementation within the VM ecosystem and then transitioned into the containerized world, where a lighter deployment model was achieved.

Keywords: High Availability, Business Continuity, Service Level Agreement (SLA), Load Balancing, Distributed Systems, Virtual Machines, Containers, CI/CD, Vagrant

This page intentionally left blank.

Acronyms

API Application Programming Interface. i

CD Continuous Delivery. i

CI Continuous Integration. i

CPU Central Processing Unit. i

DB Database. i

DCS Distributed Consensus Store. i

DNS Domain Name System. i

DR Disaster Recovery. i

FPM FastCGI Process Manager. i

GUI Graphical User Interface. i

HA High Availability. i

HTTP Hypertext Transfer Protocol. i

IaaS Infrastructure as a Service. i

IaC Infrastructure as Code. i

IP Internet Protocol. i

JSON JavaScript Object Notation. i

JWT JSON Web Token. i

OS Operating System. i

PHP PHP: Hypertext Preprocessor. i

RAM Random Access Memory. i

S3 Simple Storage Service. i

SLA Service Level Agreement. i

SPOF Single Point of Failure. i

TCP Transmission Control Protocol. i

VIP Virtual IP. i

VM Virtual Machine. i

VRRP Virtual Router Redundancy Protocol. i

WSL Windows Subsystem for Linux. i

Contents

1	Introduction	1
1.1	Context	1
1.2	Objectives	2
1.3	Document Structure	2
2	Project O	5
2.1	Original Monolithic Architecture Critique	5
2.1.1	Baseline Project Stack	5
2.1.2	Monolithic Architecture Limitations	6
2.1.3	Stateful and Stateless Layers	7
2.2	Testing Methodology and Procedures	7
2.3	Performance Metrics Results	8
2.4	Proposed HA Architecture and Tool Justification	9
2.4.1	Incremental Decoupling Roadmap	9
2.4.2	Phase 1: Load-Balanced Baseline Stack	10
2.4.3	Phase 2: PostgreSQL High Availability with Patroni and Consul . .	11
2.4.4	Phase 3: Database Connection Pooling with PgBouncer	13
2.4.5	Phase 4: Highly Available Sessions with Redis Sentinel	14
2.4.6	Phase 5: WebSocket High Availability with RatchetPHP and Redis Pub/Sub	15
2.4.7	Phase 6: Dedicated Object Storage with MinIO	18
2.4.8	Phase 7: Real-Time Monitoring with Netdata	19
2.4.9	Phase 8: Centralized Logging with Loki and Grafana	19
2.4.10	Performance Evolution Across Phases	20
2.4.11	Active-Passive Load Balancing: Nginx and Keepalived	20
2.4.12	Stateless Session Store and Cache: Redis Sentinel vs. JWT	21
2.4.13	On-Premises Object Storage: MinIO vs. NFS	21
2.4.14	Database Clustering and Consensus: Patroni, Consul, and PgBouncer	22
2.5	SLA Impact and Complexity Trade-off	22
3	Docker Containerization	25
3.1	Introduction	25
3.2	Phase 2 Objectives	26

3.3	Architecture Comparison: Virtual Machines and Containers	26
3.4	Docker Swarm Fundamentals	27
3.5	Containerized Service Mapping	28
3.6	Containerized Logging with Loki, Promtail, and Grafana	29
3.7	Networking Changes	30
3.8	Self-Healing and Recovery	30
3.9	Auto-Scaling Analysis	31
3.10	Performance and Failover Observations	31
3.11	Advantages and Disadvantages	32
4	Cloud Deployment Alternative	35
4.1	Introduction	35
4.2	Proposed GCP-Based Architecture	35
4.3	Container-Oriented Deployment Model	37
4.4	Expected Benefits and Trade-Offs	37
5	Conclusion	39
A	Report given initially	43
B	Attachments	53
B.1	Local Gallery Directory Inspection	53
B.2	Local PHP Session Directory Inspection	54
B.3	Network and Process Inspection	54
B.4	Vegeta Load Test and Plot Generation	54
B.5	Vegeta Performance Results	54
B.6	Apache Worker Exhaustion Log	55
B.7	Resource Monitoring and Log Auditing	55

List of Figures

2.1	Phase 1 request flow with Keepalived-managed VIP failover over two still-stateful application VMs.	10
2.2	Phase 2 database request flow with Patroni, Consul, HAProxy, and PgBouncer.	11
2.3	Phase 2 PostgreSQL failover sequence coordinated by Consul and Patroni.	12
2.4	Phase 3 connection-pooling flow with PgBouncer reducing PostgreSQL connection pressure.	13
2.5	Phase 4 Redis Sentinel topology for highly available PHP sessions and WebSocket Redis discovery.	14
2.6	Phase 5 WebSocket horizontal scaling using RatchetPHP instances synchronized through Redis Pub/Sub.	15
2.7	Phase 6 object-storage flow with uploaded files externalized from web-node disks to MinIO.	18
2.8	Phase 7 real-time monitoring flow with Netdata agents streaming metrics to a parent dashboard.	19
2.9	Phase 8 centralized logging flow with per-VM logs collected into Loki and explored through Grafana.	19
3.1	Docker logging ingestion with a single Promtail container reading shared Docker logs and pushing them to Loki for Grafana exploration.	29
4.1	Proposed GCP cloud alternative using Cloud Run containers and managed stateful services.	36
B.1	Architecture of the base project.	53
B.2	Initial stack image upload stored in the local gallery directory.	53
B.3	Proposed high-availability architecture.	55
B.4	Successful Jelbrek execution.	56

This page intentionally left blank.

Chapter 1

Introduction

1.1 Context

Nowadays, organizations operate in a world increasingly shaped by technology. Digital platforms are no longer only support tools; they are often the primary way enterprises present themselves, sell their products, interact with customers, and remain competitive. As this technological environment continues to evolve, companies become more dependent on the continuous availability of their applications. In this context, downtime directly represents business loss, whether through interrupted sales, unavailable services, damaged user trust, or reduced operational continuity. Therefore, modern web applications must be designed to withstand hardware failures, network partitions, and volatile traffic spikes. Traditional monolithic architectures—often implemented as "all-in-one" systems—house the reverse proxy, application runtime, database management system, and persistent storage within a single operating system instance. While this monolithic design simplifies initial development, it introduces a critical Single Point of Failure (SPOF) and creates an insurmountable barrier to horizontal scalability due to tight coupling with local state.

This laboratory project is part of the practical curriculum for the Cloud Computing and Virtualization course. The baseline project, designated "Project O," consists of a legacy monolithic PHP web application deployed on a single Virtual Machine (VM) managed by Vagrant. The application incorporates typical web workloads, including dynamic data loading from external APIs, stateful user sessions, local file uploads, persistent relational data, and bidirectional communication via WebSockets.

In this context, this work serves as a rigorous bridge between distributed systems theory and practical systems engineering. The challenge lies in deconstructing a legacy monolith and restructuring it into a high-availability, automated, and stateless infrastructure, simulating an on-premises Infrastructure as a Service (IaaS) enterprise environment.

1.2 Objectives

The global objective of this project is to design, provision, and evaluate a resilient, highly available, and horizontally scalable implementation of “Project O” (Solution A). To achieve this, our group directed its efforts toward the following technical milestones:

- **System SLA Elevation:** Transition the application’s availability from a best-effort monolithic base (SLA of approximately 95%, equivalent to around 18 days of potential annual downtime) to a highly available distributed architecture (SLA of 99.9%, reducing the maximum annual downtime to less than 9 hours).
- **State Separation and Stateless Web Layer:** Extract all stateful elements from the application execution environment. This enables the implementation of a stateless PHP-FPM web layer capable of dynamic horizontal scaling, placed behind redundant Nginx load balancers configured with failover through the Virtual Router Redundancy Protocol (VRRP) via Keepalived.
- **Resilient Cache and Session Storage:** Replace local session storage with a highly available in-memory Redis replication cluster (Master-Replica topology monitored by Redis Sentinel / HAProxy) to centralize user sessions, cache external API payloads, and broker WebSocket events.
- **Object Storage (S3-Compatible):** Implement a highly resilient on-premises MinIO object storage cluster to manage user file uploads. This replaces legacy shared file systems (such as NFS) and aligns the project with modern cloud-native storage standards.
- **Automated Database Clustering with Consensus:** Establish a highly available PostgreSQL database cluster managed by Patroni to automate Master election and failover. This configuration utilizes HashiCorp Consul as a Distributed Consensus Store (DCS) to guarantee split-brain mitigation, and integrates PgBouncer for client connection pooling, optimizing RAM utilization.
- **Infrastructure as Code (IaC) Automation:** Fully automate the provisioning, networking, and security configurations of the multi-VM topology using modular Ansible playbooks, ensuring complete reproducibility of the environment.

1.3 Document Structure

The remainder of this report is organized as follows. Chapter 2 analyzes the original *Project O* architecture, identifies its main limitations, presents the testing methodology and performance results, and explains the incremental transition toward the proposed high-availability design. Chapter 3 discusses the containerization phase, comparing the VM architecture with a Docker-based approach and evaluating the benefits and trade-offs

of using containers for deployment, service discovery, recovery, and scalability. Chapter 4 presents a cloud deployment alternative based on Google Cloud Platform, showing how the VM and container concepts could be mapped to managed services such as Cloud Run, Cloud SQL, Cloud Storage, Memorystore, Pub/Sub, and Cloud Monitoring. The final chapter summarizes the main conclusions and reflects on the limitations and possible future work.

This page intentionally left blank.

Chapter 2

Project O

2.1 Original Monolithic Architecture Critique

Before analyzing the architectural limitations, it is important to outline the main functional capabilities of "Project O". The application is a standard dynamic web platform, whose base architecture is shown in Figure B.1, and is composed of the following features:

1. **Message Forum (Database):** Users can read, insert, and delete messages, interacting directly with the PostgreSQL database.
2. **Image Gallery (File Uploads):** Users can upload .jpg images to the server, where they are stored locally and displayed in a grid.
3. **User Sessions:** A simple authentication simulation allows users to save their name, which is persisted across pages through PHP sessions.
4. **Real-Time Notifications:** A WebSocket implementation broadcasts messages to connected clients in real time.
5. **External API Integration:** The application retrieves external data, such as cat images, using JavaScript libraries.

2.1.1 Baseline Project Stack

The baseline project provided for this analysis is implemented as a Vagrant-based virtual environment. Its technological stack is composed of Ubuntu 22.04 LTS (*bento/ubuntu-22.04*) as the operating system, *Apache2* as the web server, *PHP 8.1* as the application layer, *PostgreSQL 14* as the relational database, and *Ratchet*, a PHP library, for WebSocket communication.

2.1.2 Monolithic Architecture Limitations

This initial version of represents a classic single-node monolithic architecture. All application layers – the HTTP server, the PHP runtime, the relational database, local file storage, and the WebSocket daemon – share the same kernel, CPU, RAM, and disk on the same VM.

Through testing, file-system inspection, and architectural analysis, several critical limitations were identified. These problems inherently prevent the current application from achieving high availability or horizontal scalability:

1. **Single Point of Failure (SPOF) and Resource Contention:** The PostgreSQL database runs on the same instance as the Apache web server. Network inspection confirmed that the database is bound locally to `127.0.0.1:5432`. Under heavy load, the database and web server compete for the same CPU and memory resources. Additionally, a failure of this single VM makes the entire system unavailable and may cause data loss.
2. **State Entrapment (File Uploads):** User-submitted images are stored directly on the local file system under `/vagrant/app/public_html/gallery/`. Directory inspection during execution confirmed the local persistence of uploaded files: The terminal output in Appendix B.1 confirms the presence of uploaded files in this local directory. In a horizontally scaled environment with a load balancer, instances would have desynchronized file systems, resulting in inaccessible or missing images depending on which node handled the request. The local image-storage behavior is documented in Figure B.2. Therefore, a distributed storage solution is mandatory.
3. **Local Session Management:** PHP sessions are currently stored locally under `/var/lib/php/sessions`. Terminal inspection confirmed the creation of local session files associated with the specific VM: The terminal output in Appendix B.2 confirms the creation of local session files associated with this VM. This prevents transparent load balancing, because subsequent requests from the same user routed to a different VM would not receive the active session. Migrating session state to a centralized in-memory cache is required.
4. **Isolated and Inactive WebSocket Service:** The Ratchet WebSocket server is configured in the code to listen on port 8000. However, network analysis showed that the service is not automatically started during the original provisioning process. Even if started manually, it keeps connection state in local RAM. In a distributed scenario, users connected to VM1 would be isolated from users connected to VM2, making an inter-VM Pub/Sub mechanism, such as Redis, necessary.
5. **Imperative Provisioning Script:** The current infrastructure is provisioned through a single sequential Bash script, `web.sh`. This approach lacks idempotency, meaning the script cannot be safely executed multiple times without causing errors or undesired states. It also mixes package installation, database configuration, and

application deployment in one file, making maintenance and scalability difficult. A transition to a declarative configuration-management tool, such as Ansible, is required for reliable Infrastructure-as-Code (IaC) deployment.

2.1.3 Stateful and Stateless Layers

Before explaining the methodology used to decouple the application layers, it is important to understand the difference between *stateful* and *stateless* layers [2]. A stateless layer does not keep unique user or application data locally between requests. If one stateless web node fails, another equivalent node can process the next request because the required state is stored elsewhere. This makes stateless components easier to replicate, load balance, restart, and scale horizontally.

A stateful layer, by contrast, stores data that must persist or remain consistent over time. Databases, user sessions, uploaded files, caches, and WebSocket connection metadata are examples of stateful concerns. These layers cannot simply be duplicated without coordination, because inconsistent state would lead to lost sessions, missing uploads, stale data, or split-brain situations.

For this reason, the high-availability strategy followed in this report consists of making the web/application tier as stateless as possible while moving stateful responsibilities into specialized and resilient services. PostgreSQL handles relational data, Redis handles sessions and Pub/Sub coordination, MinIO handles uploaded objects, and Consul coordinates database leadership. This separation is what allows the application tier to scale horizontally while preserving consistency in the stateful layers.

2.2 Testing Methodology and Procedures

The testing methodology was guided by the need to identify which components made the baseline application less redundant and less performant, and which technologies were best suited to address each limitation. Rather than testing the final architecture only once, the evaluation followed an incremental decoupling approach: starting from the original monolithic Vagrant project, each architectural layer was progressively separated into a dedicated tier and then evaluated.

This approach made it possible to observe how the application became more redundant and scalable as each layer was removed from the baseline VM. Load tests were executed with the assistance of *Vegeta*, but the main objective was not only to generate traffic; it was to compare the behavior of the system before and after each architectural change. In this way, the test results supported the selection of technologies such as load balancing, centralized sessions, distributed storage, and database clustering.

As shown throughout this report, the first step was to decouple the load-balancing tier. By introducing a dedicated load balancer, requests could be distributed across more than one web node, reducing CPU and memory pressure on a single server and creating

the foundation for horizontal scaling. The same logic was then applied to the remaining stateful layers: sessions, file uploads, WebSockets, and the database.

The evaluation process consisted of the following steps:

1. **Baseline Inspection:** The original Vagrant-based application was inspected to understand how the web server, PHP runtime, PostgreSQL database, file uploads, sessions, and WebSocket service were coupled inside a single VM. The local persistence of PHP sessions and uploaded images was verified through file-system inspection, with supporting terminal evidence documented in Appendix B.2 and Appendix B.1.
2. **Incremental Tier Decoupling:** Each major component was analyzed as a candidate for separation into its own tier. The load-balancing tier was considered first, because distributing requests across multiple web nodes directly reduces resource pressure on the baseline server. Subsequent decoupling focused on session storage, object storage, real-time messaging, and the database layer.
3. **Network and Process Validation:** Socket Statistics (`ss`) commands filtered with `grep` were executed inside the VM to verify port bindings and process isolation. This inspection confirmed that the WebSocket service was not listening on port 8000, while PostgreSQL was bound locally to 127.0.0.1:5432. The corresponding terminal evidence is included in Appendix B.3.
4. **Load Testing with Vegeta:** To simulate real traffic and stress the infrastructure limits, the HTTP load-testing tool *Vegeta* was executed from the WSL Ubuntu environment. The attack targeted the database-backed endpoint `http://192.168.44.10/db.php`, using a sustained rate of 200 requests per second for 60 seconds. The commands used to generate the binary results file and the interactive HTML plot are documented in Appendix B.4.
5. **Resource Monitoring and Log Auditing:** During the load tests, VM resource consumption was monitored in real time using `htop`. After each test, the Apache global error log was inspected to identify technical bottlenecks, such as connection saturation or backend failures. These monitoring and log-auditing commands are listed in Appendix B.7.

2.3 Performance Metrics Results

The load-test results confirmed that the original monolithic architecture collapses under the specified workload. With *Vegeta* configured to send 200 requests per second for 60 seconds against the database-backed endpoint, the application received 12,000 requests but successfully processed only 711 of them. The complete Vegeta output is included in Appendix B.5.

The main observations were the following:

1. **Success Rate:** Only 5.92% of the requests were processed successfully with HTTP status 200. The remaining approximately 94% failed, mostly as lost, cancelled, or timed-out connections represented by status code 0.
2. **Throughput:** Although the input rate was 200 requests per second, the server achieved only 7.90 successful transactions per second. This indicates a severe bottleneck at the web-server and database layers.
3. **Latency:** The average response time increased to 28.9 seconds, while the 90th, 95th, and 99th percentiles reached the 30-second client timeout limit. This behavior confirms that the system was not merely slowing down, but saturating until requests expired.
4. **Apache Worker Exhaustion:** The Apache error log revealed the direct cause of the collapse: the server reached its `MaxRequestWorkers` limit during the load test. The relevant log entry is documented in Appendix [B.6](#).

These results show that the 94% failure rate and client timeouts were not primarily caused by internal application errors, such as HTTP 500 responses. Instead, they were caused by the exhaustion of Apache’s request-processing capacity. This inability to scale vertically under the current configuration strongly justifies the introduction of a load-balancing tier to distribute incoming connections across multiple web nodes.

2.4 Proposed HA Architecture and Tool Justification

After group discussion, the main objective became reaching the proposed high-availability architecture shown in Figure [B.3](#). The proposed solution therefore adopts a decoupled, multi-tier infrastructure designed to improve availability, scalability, and fault isolation within the constraints of an on-premises virtualized laboratory environment. The following subsections justify the selected technologies for each architectural layer and explain how they contribute to the target high-availability design.

2.4.1 Incremental Decoupling Roadmap

The transition from the baseline monolith to the proposed high-availability architecture was performed incrementally rather than as a single redesign. This approach made each architectural decision easier to validate: after the implementation of each layer, manual functional tests and *Vegeta* load tests were always performed to confirm that the new component improved resilience without degrading performance.

Table [2.1](#) summarizes the implementation path followed by the group. Each phase adds one specific capability and prepares the infrastructure for the next layer of decoupling.

Phase	Layer Added	Resilience or Performance Gained	VMs
1	Redundant load balancers with Keepalived and Nginx	Traffic distribution across two still-stateful application VMs	4
2	Patroni and Consul	Automatic PostgreSQL failover through leader election	7
3	PgBouncer	Connection pooling, reducing PostgreSQL connection pressure by approximately 85%	7
4	Redis Sentinel cluster	Automatic PHP session failover with Redis master promotion	9
5	RatchetPHP with Redis Pub/Sub	Cross-instance real-time messaging between WebSocket servers	9
6	Dedicated MinIO VM	Separation of object storage from Redis and local web-node disks	10
7	Netdata monitoring	Real-time monitoring dashboard for the complete VM fleet	11
8	Loki and Grafana logging	Centralized logs with per-VM labels and interactive exploration	11

Table 2.1: Incremental phases used to decouple the baseline architecture.

2.4.2 Phase 1: Load-Balanced Baseline Stack

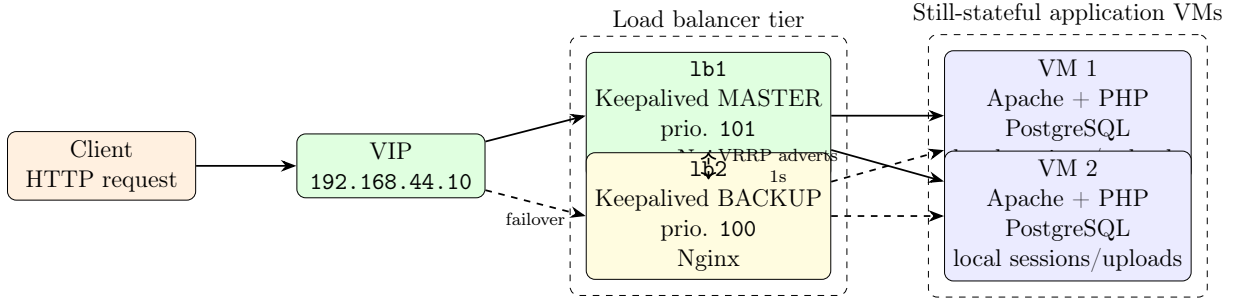


Figure 2.1: Phase 1 request flow with Keepalived-managed VIP failover over two still-stateful application VMs.

The first implementation phase introduced only the load-balancing tier in front of two application VMs. At this stage, the objective was not yet to decouple the database, sessions, uploads, or other stateful services. Instead, both backend VMs still ran the remaining monolithic stack locally, including Apache, PHP, PostgreSQL, local sessions, and local uploaded data. This phase therefore validated the first step toward horizontal scaling: distributing client traffic across two equivalent but still stateful application nodes.

In this phase, client requests entered the system through the virtual IP (VIP)

192.168.44.10, managed by *Keepalived* across the two load balancers. The configuration was implemented through an Ansible role composed of a VRRP template, deployment tasks, and a restart handler. In the template, **1b1** is configured as **MASTER** with priority 101, while **1b2** is configured as **BACKUP** with priority 100. Both nodes use the **eth1** interface, the same **virtual_router_id**, and the same authentication password, allowing the VIP to float between them through VRRP advertisements sent every second. During normal operation, **1b1** owns the VIP and receives HTTP traffic through *Nginx*, which distributes requests across the two backend VMs. If **1b1** stops sending VRRP advertisements, **1b2** detects the failure after approximately three missed advertisements, promotes itself to **MASTER**, and assumes 192.168.44.10. This preserved the same client-facing address and removed the load balancer as a single point of failure, but the backend application VMs still retained their own local state.

The main alternative would have been to scale the original VM vertically by increasing CPU and memory. This was rejected because it would keep a single traffic entry point and would only postpone the bottleneck. The load-balanced baseline performed significantly better than the original monolith in the manual and *Vegeta* tests, but it was still not a fully stateless or highly available architecture because sessions, uploaded files, and database state remained inside the backend VMs. This limitation motivated the following phases, where each stateful responsibility was progressively extracted into a dedicated layer.

2.4.3 Phase 2: PostgreSQL High Availability with Patroni and Consul

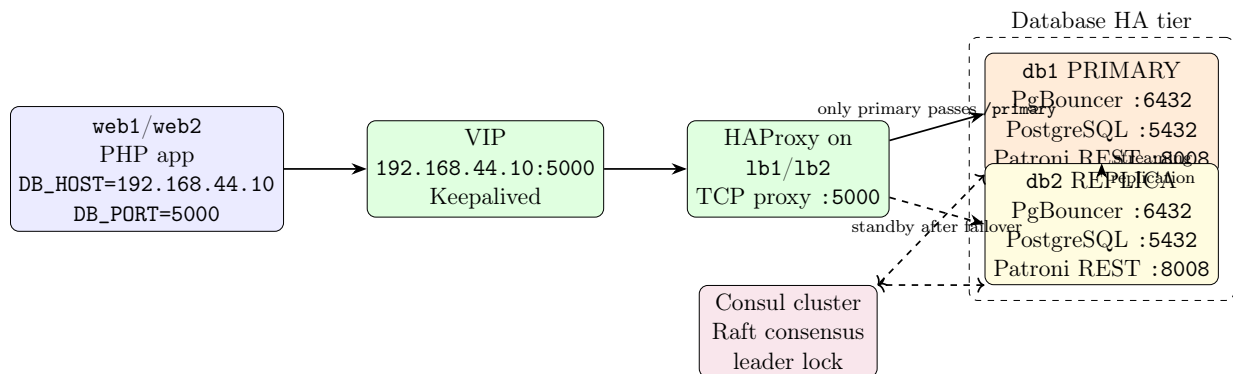


Figure 2.2: Phase 2 database request flow with Patroni, Consul, HAProxy, and PgBouncer.

After the initial load-balancing phase, the second phase separated the database tier from the backend application VMs and made PostgreSQL highly available. The objective was to ensure that the PHP application no longer connected to a local database instance. Instead, the application configuration pointed to the shared VIP, 192.168.44.10, on port 5000. This address is served by HAProxy on the active load balancer, which forwards database traffic only to the current PostgreSQL primary.

The database layer was implemented with *Patroni*, *Consul*, *HAProxy*, and *PgBouncer*. Patroni runs on both database VMs and manages the PostgreSQL role of each node. Consul runs as the distributed consensus layer and stores the leader lock used by Patroni to decide which database node is allowed to become primary. PgBouncer is colocated with each PostgreSQL node and listens on port 6432, reducing direct connection pressure on PostgreSQL. HAProxy runs on the load balancers and exposes port 5000, checking Patroni's REST API on port 8008 before forwarding traffic to PgBouncer on the primary database node.

In the Patroni configuration, both database nodes belong to the same cluster scope, `project_o_cluster`. Patroni exposes a REST API on `:8008`, connects to the local Consul agent on `127.0.0.1:8500`, and manages PostgreSQL on `:5432`. The distributed configuration uses a `ttl` of 30 seconds, `loop_wait` of 10 seconds, `retry_timeout` of 10 seconds, and a maximum failover lag of 1 MB. These values define how long the leader lock remains valid, how frequently Patroni checks cluster state, and which replicas are safe candidates for promotion.

Consul was configured as a two-node server cluster on `db1` and `db2`, using the addresses `192.168.44.31` and `192.168.44.32`. Its role is to provide Raft-backed coordination for leader election. When `db1` is primary, it holds the leader key under the PostgreSQL namespace in Consul. If `db1` fails or loses the ability to renew its session, the lock expires and `db2` can safely acquire it. This prevents split-brain because only the node holding the Consul lock is allowed by Patroni to accept writes as primary.

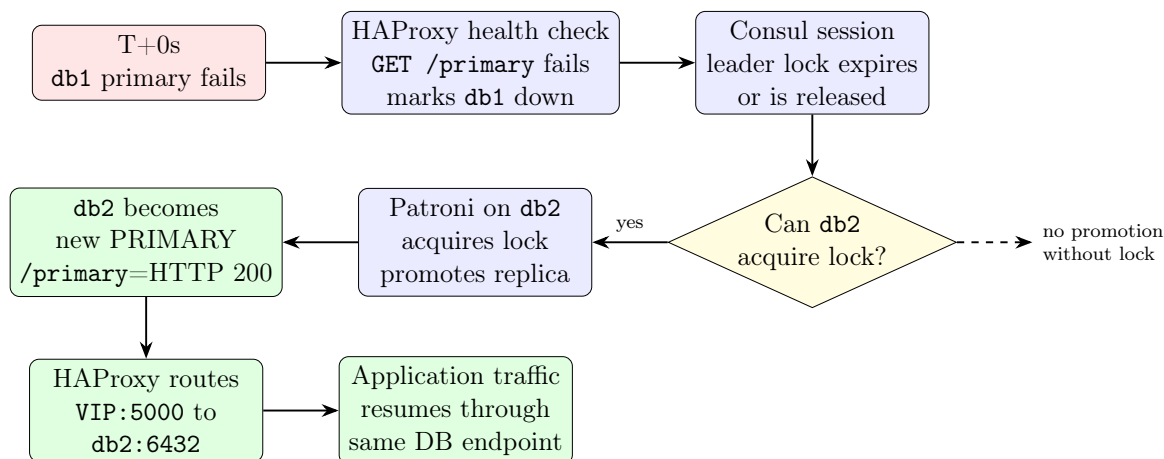


Figure 2.3: Phase 2 PostgreSQL failover sequence coordinated by Consul and Patroni.

HAProxy performs the final routing decision. Its database frontend listens on `:5000`, while the backend performs HTTP health checks against Patroni using `GET /primary` on port 8008. A primary node returns HTTP 200, while a replica returns HTTP 503. Therefore, HAProxy forwards TCP connections only to the PgBouncer instance on the current primary, avoiding accidental writes to a replica. The PHP application reaches the database through VIP:5000, not by connecting directly to either PostgreSQL VM.

The failover test confirmed the behavior of this design. During the test, *Vegeta* generated 10 requests per second for 60 seconds while the primary database node was stopped. HAProxy first marked the old primary as unavailable, Patroni promoted the replica after the Consul leader lock changed, and HAProxy then resumed forwarding traffic to the new primary. The measured promotion completed in approximately four seconds, and the test completed with 600 successful requests out of 600, corresponding to 100% success and zero failed requests. This proved that the database layer could survive a primary-node failure while preserving a stable application-facing database endpoint.

2.4.4 Phase 3: Database Connection Pooling with PgBouncer

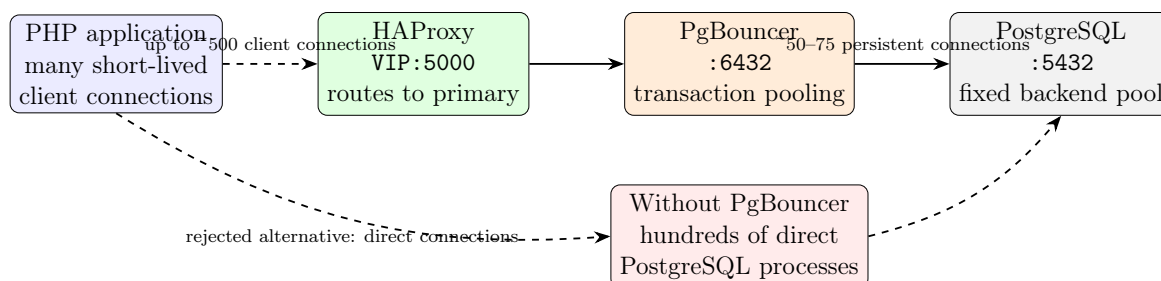


Figure 2.4: Phase 3 connection-pooling flow with PgBouncer reducing PostgreSQL connection pressure.

Once database failover was functional, the next bottleneck was connection management. PHP applications frequently open and close database connections, and PostgreSQL uses a process-per-connection model. Under high request rates, this can waste CPU and RAM on connection handling instead of query execution.

To address this, *PgBouncer* was introduced between HAProxy and PostgreSQL. PgBouncer runs close to the database nodes and maintains a pool of persistent backend connections. The PHP application can still create many short-lived client connections, but these are multiplexed onto a much smaller number of PostgreSQL connections.

The alternative was to keep direct PostgreSQL connections and simply raise `max_connections`, but that would increase memory usage and make the database more vulnerable under spikes. Another option was application-level persistent connections, but that would require more intrusive changes in the PHP code. PgBouncer was selected because it solved the issue transparently at the infrastructure layer. At 500 requests per second, it reduced PostgreSQL connection pressure from roughly 500 direct connections to approximately 50–75 pooled connections, an improvement of about 85%. It preserved 100% success, kept mean latency at 2.03 ms, and improved p99 latency from 4.99 ms to 3.61 ms.

2.4.5 Phase 4: Highly Available Sessions with Redis Sentinel

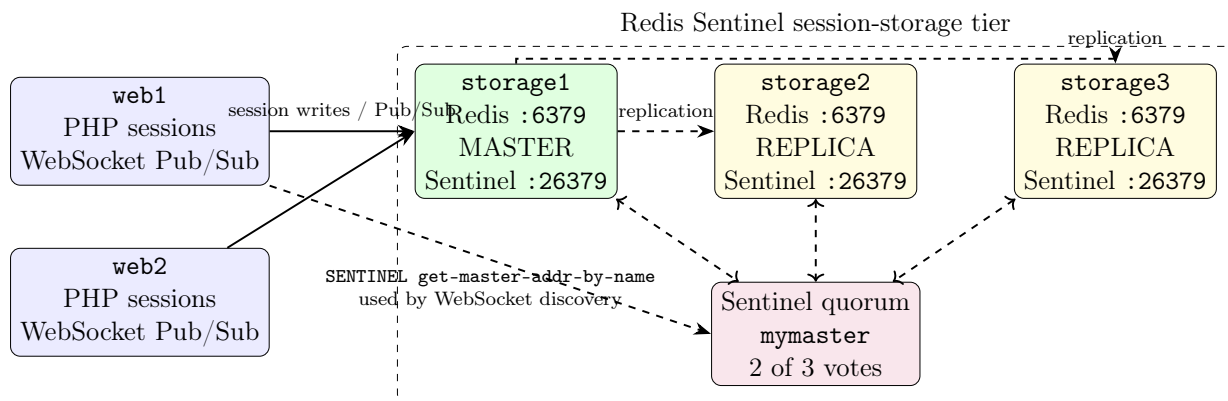


Figure 2.5: Phase 4 Redis Sentinel topology for highly available PHP sessions and WebSocket Redis discovery.

The original application stored PHP sessions locally, and the first distributed version still depended on a single Redis instance. This created a new stateful single point of failure: if Redis failed, users could lose active sessions and WebSocket coordination through Redis Pub/Sub would also become unavailable. The objective of this phase was therefore to remove the Redis single point of failure by expanding it into a Sentinel-managed topology.

The implemented Redis layer uses three storage VMs: **storage1** at 192.168.44.41, **storage2** at 192.168.44.42, and **storage3** at 192.168.44.43. Redis initially runs with **storage1** as the master, while **storage2** and **storage3** are configured as replicas through `replicaof 192.168.44.41 6379`. Redis is configured to listen on the internal network, protected mode is disabled for the laboratory environment, and authentication is enabled with `redispass`. This gives the system one writable Redis master and two synchronized replicas that can be promoted during failure.

Redis Sentinel was then deployed on the three storage VMs. Each Sentinel listens on port 26379 and monitors the master name **mymaster**. The configuration uses `sentinel monitor mymaster 192.168.44.41 6379 2`, where the final value defines a quorum of two Sentinels. A single Sentinel can mark the master as subjectively down after 5000 ms without a response, but failover only starts when at least two Sentinels agree that the master is objectively down. The failover timeout is set to 30000 ms, and `parallel-syncs` is set to 1, so replicas are resynchronized carefully after promotion.

On the application side, PHP session storage was moved from local files to Redis by installing the PHP Redis extension and setting `session.save_handler = redis`. In the implemented configuration, the PHP session path points to a Redis endpoint with authentication. A more resilient variant is also possible by pointing PHP directly to Sentinel so that the current master is discovered automatically. For WebSockets, the server uses the Sentinel command `SENTINEL get-master-addr-by-name mymaster` against the available Sentinel nodes. This avoids hardcoding the Redis master IP in the WebSocket code and

allows the server to reconnect to the promoted master after a failover.

The failover process starts when the Redis master on `storage1` stops responding. After five seconds, Sentinels mark it as subjectively down; once two out of three Sentinels agree, the master is considered objectively down and failover begins. A Sentinel leader selects the best replica, promotes it with `SLAVEOF NO ONE`, reconfigures the remaining replica to follow the new master, and updates `mymaster` to the new Redis address. In the observed test, `storage2` became the new master after approximately five to ten seconds.

Alternative approaches included sticky sessions at the load balancer, JWT-based sessions, Redis Cluster, and Keepalived in front of Redis. Sticky sessions were rejected because they do not solve backend failure and still bind a user to a specific node. JWTs would require significant changes to the legacy PHP application and make immediate session revocation harder. Redis Cluster was unnecessary because the project needed high availability for session state, not sharding. Keepalived would only move an IP address and would not automatically promote a Redis replica. Redis Sentinel was selected because it provides automatic master monitoring, quorum-based failover, and replica promotion with relatively low complexity.

During failover testing, *Vegeta* generated 10 requests per second for 60 seconds. The system completed 600 successful requests out of 600, corresponding to 100% success and zero failed requests. The mean latency was 2.61 ms, and the maximum latency reached 12.69 ms during the failover window. This confirmed that Redis Sentinel removed the Redis single point of failure while preserving session availability during master promotion.

2.4.6 Phase 5: WebSocket High Availability with RatchetPHP and Redis Pub/Sub

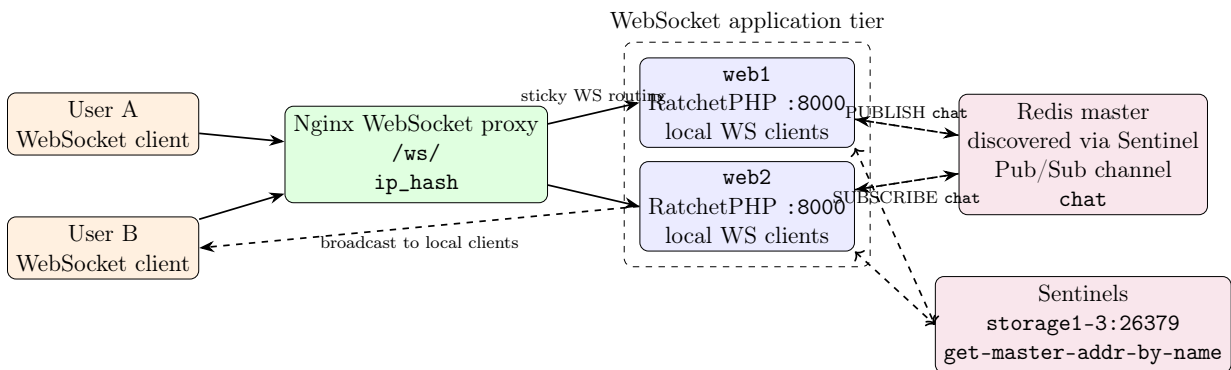


Figure 2.6: Phase 5 WebSocket horizontal scaling using RatchetPHP instances synchronized through Redis Pub/Sub.

WebSockets introduce a different type of state because connections are persistent and remain bound to the server that accepted them. Running RatchetPHP independently on multiple web nodes would create isolated real-time islands: users connected to `extttweb1`

would not automatically receive events generated on `extttweb2`. The objective of this phase was therefore to synchronize the RatchetPHP instances running on `extttweb1` and `extttweb2` through Redis Pub/Sub. In this model, a message received by one instance is published once to Redis and then delivered by all RatchetPHP instances to their own locally connected clients.

The implementation is distributed across the infrastructure automation files rather than being configured manually. The `extttansible/roles/websocket/tasks/main.yml` role creates the WebSocket directory under `exttt/var/www/html/ws`, installs RatchetPHP and the Redis client dependencies through Composer, deploys the templated server from `extttansible/roles/websocket/templates/websockets_extunderscore_server.php.j2`, and registers `extttratchet.service` as a systemd unit. The handler in `extttansible/roles/websocket/handlers/main.yml` restarts the service whenever the template changes. The role assignment in `extttansible/site.yml` co-locates the WebSocket role with the existing `extttwebservers` group, while the Nginx role on the load balancers provides the reverse proxy for `exttt/ws/`. The VM topology is defined through the `extttVagrantfile` and `extttansible/inventory.ini`: `extttweb1` and `extttweb2` run RatchetPHP on `exttt192.168.44.21:8000` and `exttt192.168.44.22:8000`, while `extttstorage1`, `extttstorage2`, and `extttstorage3` host the Redis Sentinel layer on `exttt192.168.44.41-43:26379`.

The WebSocket server does not rely on a fixed Redis master IP. At startup, it queries the Sentinel endpoints `exttt192.168.44.41:26379`, `exttt192.168.44.42:26379`, and `exttt192.168.44.43:26379` with `extttSENTINEL get-master-addr-by-name mymaster`. After discovering the current Redis master, each Ratchet instance opens a publisher connection authenticated with the Redis password and publishes application messages to the shared channel `extttchat`. This design reuses the highly available Redis Sentinel layer introduced in the previous phase instead of creating a separate messaging service for WebSockets.

Inside the RatchetPHP server, the `extttNotificationServer` class maintains local WebSocket clients with `extttSplObjectStorage`. When a client opens a connection, the server extracts the optional `extttuser_extunderscore_id` from the query string, registers the connection locally, and stores metadata in Redis. The hash `extttws:connections` maps each `extttresourceId` to information such as server identifier, user identifier, and connection time, while `extttws:users` maps a user identifier to the current connection resource. When a client sends a message, the server enriches it with a timestamp and sender identifier, sanitizes the message field with `exttthtmlspecialchars`, and publishes the JSON payload to the Redis `extttchat` channel.

For receiving cross-instance messages, each Ratchet server also runs a subscriber path. The subscriber listens to the Redis `extttchat` channel and passes received messages back into the Ratchet event loop, where they are sent to all local WebSocket clients attached to that specific server. In the implemented template this is achieved with a child process that performs the blocking Redis `extttSUBSCRIBE`, writes received messages to a temporary IPC file, and a ReactPHP periodic timer that checks for new messages every 50 ms. Therefore, if User A is connected to `extttweb1` and sends a message, `extttweb1` publishes it to Redis; Redis delivers it to the subscribers on both `extttweb1` and `extttweb2`; and

extttweb2 broadcasts it to User B even though User B is connected to a different backend.

Nginx on the load balancers proxies WebSocket traffic through the `exttt/ws/` location. The WebSocket upstream uses `extttip extunderscore` hash, so the same client IP is consistently routed to the same Ratchet backend after the initial handshake. The proxy also enables HTTP/1.1 upgrade headers with `extttUpgrade` and `extttConnection: upgrade`, forwards the original host and client IP, and uses long read/write timeouts to avoid interrupting persistent WebSocket sessions. However, `extttip extunderscore` hash is not what provides cross-node message consistency; it only keeps a single connection stable. The actual horizontal synchronization is provided by Redis Pub/Sub.

The complete message flow is as follows: User A connects through Nginx and is routed to `extttweb1`; `extttweb1` receives the WebSocket message in `exttttonMessage()`; `extttweb1` publishes the JSON payload to Redis on channel `extttchat`; Redis forwards the payload to every subscribed Ratchet instance; finally, `extttweb1` and `extttweb2` each broadcast the message to their own local clients. Without Pub/Sub, `extttweb1` would only deliver the event to its local clients and users on `extttweb2` would never receive it. With Pub/Sub, real-time delivery scales horizontally across the web tier.

The alternative was to create dedicated WebSocket VMs, but this would add infrastructure without solving the need for a shared messaging backplane. Sticky sessions alone were also insufficient because users connected to different nodes would still remain isolated. Running WebSockets without Redis would scale the number of connections but not the real-time functionality. Co-locating RatchetPHP on `extttweb1` and `extttweb2` reused the existing web tier, while Redis Pub/Sub provided the missing cross-instance synchronization.

This phase did not target raw HTTP throughput; its performance gain was functional. Real-time notifications became horizontally scalable, and users connected to different web nodes could receive the same broadcast events. Validation consisted of checking that `extttratchet.service` was active on both web nodes, that both instances listened on port `exttt8000`, that the web nodes had Redis connections on port `exttt6379`, that Redis reported subscribers on the `extttchat` channel with `extttPUBSUB NUMSUB chat`, and that publishing a test JSON message through `extttredis-cli PUBLISH chat` was delivered through the connected WebSocket clients. Additional validation also inspected `extttHGETALL ws:connections` and `extttHLEN ws:connections` to confirm that active WebSocket connection metadata was being stored centrally.

2.4.7 Phase 6: Dedicated Object Storage with MinIO

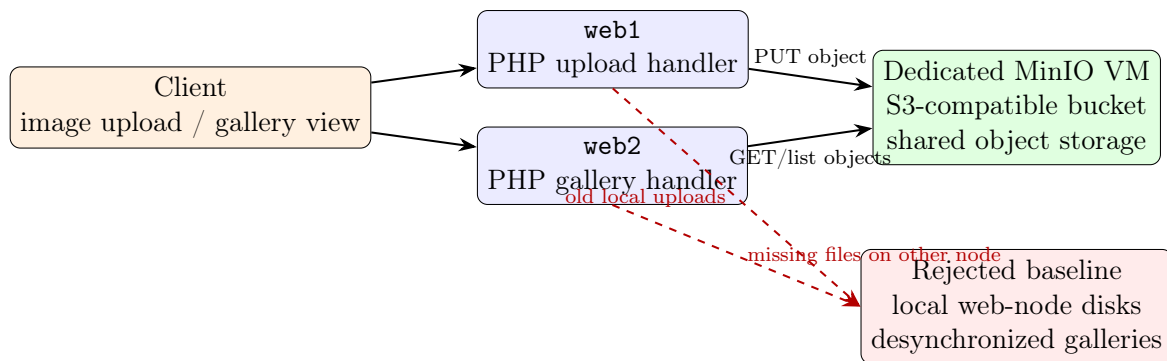


Figure 2.7: Phase 6 object-storage flow with uploaded files externalized from web-node disks to MinIO.

File uploads were another source of stateful coupling. In the baseline application, uploaded images were stored on the local web-node file system. In a horizontally scaled environment, this causes inconsistent galleries because each web node has a different local disk.

MinIO was introduced as an S3-compatible object storage layer, and later moved to a dedicated VM. This separation removed uploaded files from the web nodes and avoided mixing MinIO with Redis on the same storage VM. Web nodes can mount or access the shared object storage consistently, ensuring that uploaded images remain available regardless of which web server handles the request.

A traditional alternative would have been NFS. However, NFS would introduce a central shared-filesystem dependency, locking concerns, and weaker alignment with cloud-native storage patterns. Keeping MinIO on the same VM as Redis was also rejected because Redis is memory-oriented while object storage is disk and network-oriented. Moving MinIO to `minio1` increased the infrastructure to ten VMs but improved separation of concerns and reduced resource contention between session storage and uploaded media.

2.4.8 Phase 7: Real-Time Monitoring with Netdata

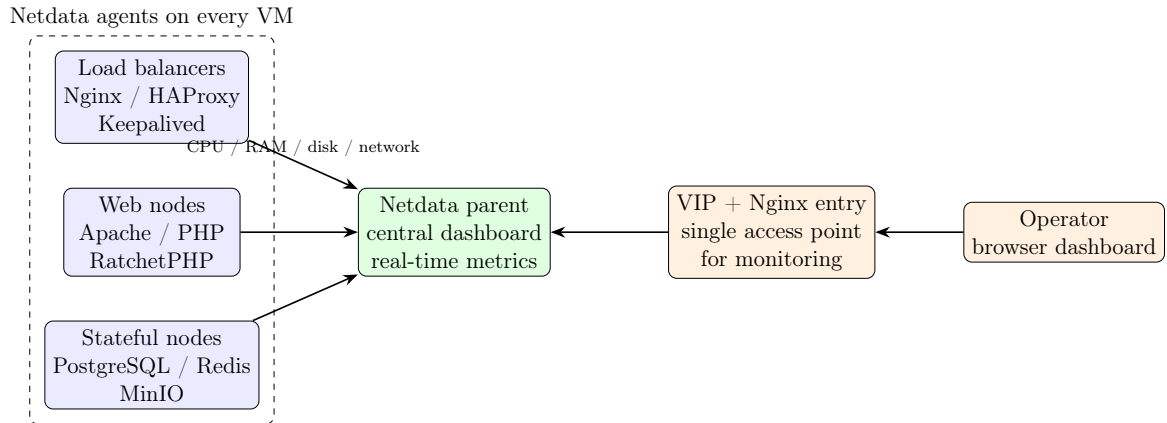


Figure 2.8: Phase 7 real-time monitoring flow with Netdata agents streaming metrics to a parent dashboard.

As the architecture grew, manual inspection through SSH became insufficient. Diagnosing issues across load balancers, web nodes, database nodes, storage nodes, and MinIO required a centralized observability interface.

Netdata was added as a real-time monitoring layer. Lightweight agents collect CPU, memory, disk, network, and process metrics from each VM and stream them to a parent monitoring node. The dashboard is exposed through the same VIP and Nginx entry point, allowing the group to observe the health of the complete infrastructure from one place.

The alternative was to continue using manual commands such as `htop`, `df`, and `journalctl` on each VM. This was not scalable for an 11-VM environment and would make incident response reactive. Netdata did not improve request latency directly, but it improved operational performance by reducing diagnostic time and making saturation visible before it became an outage.

2.4.9 Phase 8: Centralized Logging with Loki and Grafana

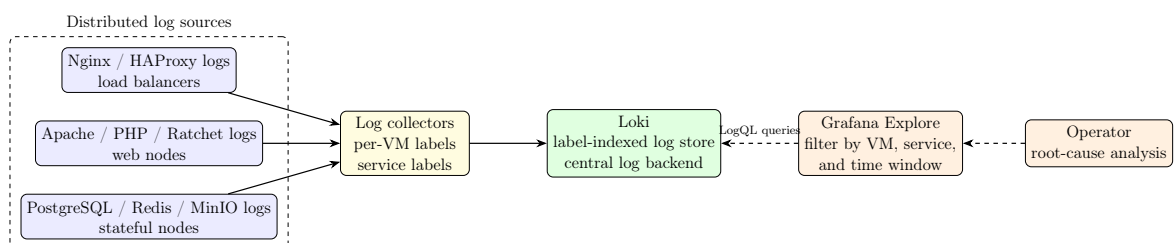


Figure 2.9: Phase 8 centralized logging flow with per-VM logs collected into Loki and explored through Grafana.

The final observability improvement was centralized logging. While Netdata provides metrics, troubleshooting distributed systems also requires correlated logs from multiple machines. Without centralized logging, each failure investigation requires manually checking logs on every VM.

Loki and Grafana were introduced to collect and explore logs with per-VM labels, following the official Loki installation and setup model described by Grafana Labs [3]. In this architecture, *Promtail* is the ingestion agent responsible for tailing local log files, attaching labels such as VM name, service name, and environment, and pushing the resulting log streams to Loki. This allows events from Nginx, Apache, PostgreSQL, Redis, MinIO, and supporting services to be searched from a single interface. Grafana Explore makes it possible to filter logs by VM, service, and time window, which is essential when diagnosing failures in a multi-tier system.

Because the prototype is VM-based rather than Kubernetes-based, Promtail must be deployed using a sidecar-like technique adapted to virtual machines. Instead of running as a Kubernetes sidecar container inside the same pod as the application, each VM runs its own lightweight Promtail companion process or service next to the workload. This per-VM sidecar pattern keeps log collection close to the source, avoids exposing local log files over the network, and ensures that each machine independently forwards its Nginx, Apache, PostgreSQL, Redis, MinIO, or system logs into Loki. In practice, Ansible can provision Promtail as a systemd-managed agent on every VM, with a configuration file that maps local log paths to consistent Loki labels.

Alternatives such as only using local log files or deploying a heavier Elasticsearch-based stack were less attractive for the laboratory constraints. Local logs do not scale operationally, while Elasticsearch would require more RAM and storage. Loki was selected because it indexes labels rather than full log contents, making it lighter for this environment.

2.4.10 Performance Evolution Across Phases

The incremental process showed that resilience could be added without sacrificing performance. The most relevant performance improvement came from PgBouncer, which reduced database connection pressure by approximately 85% while maintaining the same mean latency. Later phases focused primarily on availability, state consistency, and observability.

The key takeaway is that each phase removed one form of coupling from the original system: traffic entry, database access, connection management, sessions, real-time messaging, file uploads, metrics, and logs. This step-by-step process transformed the baseline application into the proposed high-availability architecture while keeping the system measurable at every stage.

2.4.11 Active-Passive Load Balancing: Nginx and Keepalived

To distribute client traffic across the stateless web tier, we deploy redundant *Nginx* Load Balancers. However, having a single load balancer would merely shift the SPOF up the

Phase	Rate	Success	Mean Latency	p99	VMs
Initial distributed stack	500/s	100%	2.03 ms	4.99 ms	7
PostgreSQL failover	10/s	100%	3.44 ms	–	7
With PgBouncer	500/s	100%	2.03 ms	3.61 ms	7
Redis Sentinel	10/s	100%	2.61 ms	–	9
WebSocket HA	–	Validated	–	–	9
Dedicated MinIO	–	Validated	–	–	10
Netdata and logging	–	Validated	–	–	11

Table 2.2: Performance and validation summary across the decoupling phases.

stack. To resolve this, we configure *Keepalived* using the Virtual Router Redundancy Protocol (VRRP).

A Virtual IP (VIP) is shared between both load balancer VMs. If the active Nginx node fails, Keepalived triggers a sub-second failover, automatically migrating the VIP to the passive node. This approach guarantees uninterrupted traffic routing at the entry point.

2.4.12 Stateless Session Store and Cache: Redis Sentinel vs. JWT

To transition the PHP application to a stateless paradigm, we must externalize PHP session storage. While JSON Web Tokens (JWT) were considered for pure client-side state, they would require a massive and complex rewrite of the legacy PHP code, alongside introducing security vulnerabilities such as the inability to instantly revoke active sessions.

Instead, we utilize *Redis* configured in a Master-Replica topology. To prevent Redis from becoming a new SPOF, we integrate *Redis Sentinel* combined with a local *HAProxy* layer on the load balancers. This configuration detects Master failures in seconds, automates replica promotion, and dynamically updates the destination ports without requiring code refactoring. The PHP application’s session path is simply redirected to the HAProxy IP via the ‘php.ini’ file, preserving code integrity while achieving stateless elasticity.

2.4.13 On-Premises Object Storage: MinIO vs. NFS

Traditionally, local file uploads in distributed web nodes were resolved using shared filesystems like Network File System (NFS). However, NFS suffers from severe locking issues, single-point-of-failure storage daemons, and poor performance under high concurrency.

We choose to deploy a *MinIO* cluster. MinIO provides an on-premises, S3-compatible, highly resilient object storage service. By integrating the Amazon S3 SDK into PHP-FPM, uploads are streamed directly to MinIO’s distributed bucket. MinIO handles high availability, data redundancy through erasure coding, and seamless horizontal scaling, aligning the project with modern cloud-native standards.

2.4.14 Database Clustering and Consensus: Patroni, Consul, and PgBouncer

The database layer requires the highest degree of data safety and failover automation. Simple active-passive replication with scripts (like older ‘repmgr’ setups) is highly susceptible to network partitions, which often cause a catastrophic *split-brain* scenario where two database nodes accept writes simultaneously.

To resolve this, we implement *Patroni* to manage our PostgreSQL cluster. Patroni leverages a Distributed Consensus Store (DCS), for which we deploy a *HashiCorp Consul* cluster. Patroni uses Consul’s Raft consensus algorithm to acquire a leader key. If the primary PostgreSQL node fails to renew its lease due to a network partition, it is automatically demoted by Patroni, and a healthy replica is safely promoted, mitigating the split-brain risk mathematically.

To optimize database connections, we place *PgBouncer* in front of the PostgreSQL cluster. Because PHP-FPM opens and closes database connections on every script execution, it would quickly deplete PostgreSQL’s process memory limit under load. PgBouncer acts as a lightweight connection pooler, maintaining a fixed pool of persistent backend connections and drastically reducing RAM consumption on the database VMs.

Note on backups: The group also considered adding a dedicated database backup strategy using *pgBackRest*, a tool from the PostgreSQL ecosystem designed for reliable backup and restore operations. However, due to time constraints, this backup layer was not implemented in the final prototype.

2.5 SLA Impact and Complexity Trade-off

Evaluating the system’s operational transition highlights a major architectural trade-off: **Resilience vs. Complexity**. In this context, *Service Level Agreement* (SLA) refers to the expected percentage of time that the application remains available to users during a given period, usually measured annually. For example, an SLA of 99.9% means that the system should be unavailable for no more than approximately 8.76 hours per year.

Mathematically, the monolithic baseline is characterized by a serial dependency model, leading to an estimated SLA of $\sim 95\%$ due to frequent manual maintenance and unmonitored crashes:

$$A_{\text{monolith}} \approx 95\% \quad (\sim 18.2 \text{ days of annual downtime})$$

By introducing active redundancy across all layers, our proposed infrastructure pushes individual tier availability toward 3 and 4 "Nines" (e.g., 99.9% for clustered storage and 99.99% for load balancers). However, because we run on limited local virtualized hardware (Vagrant), deploying a multi-node cluster for every single layer (such as Redis Sentinel and MinIO) consumes massive amounts of local host RAM.

If we run Redis and MinIO on single VMs with local backups (relying on fast manual or scripted recovery) due to hardware constraints, they act as serial components in our

availability chain:

$$A_{\text{total}} = A_{\text{LB}} \times A_{\text{Web}} \times A_{\text{Redis_SPOF}} \times A_{\text{MinIO_SPOF}} \times A_{\text{DB}}$$

In this constrained virtual laboratory scenario, our practical SLA will hover around 99% (**Two Nines**). This still represents an extraordinary 80% **reduction in annual downtime** compared to the monolith, despite introducing an $8\times$ increase in configuration, provisioning, and network management complexity. Additional execution evidence is included in Figure [B.4](#).

This page intentionally left blank.

Chapter 3

Docker Containerization

3.1 Introduction

The first phase of the project addressed the limitations of the original monolithic application by separating each critical responsibility into its own dedicated tier, resulting in an 11-VM high-availability stack. That solution improved redundancy by separating the load balancers, web servers, database nodes, Redis nodes, MinIO storage, and monitoring services. However, this VM-based design also introduced a significant operational cost: every tier required a complete operating-system instance, static IP planning, Ansible provisioning, service management, and manual recovery procedures when a VM failed.

The second phase explores a different virtualization model: containerization with Docker Swarm. Instead of running each service inside a separate Ubuntu virtual machine, the same architectural components are packaged as containers that share the host kernel. The goal is not to remove the high-availability mechanisms designed in Phase 1, but to reproduce them in a lighter and more declarative environment. PostgreSQL is still managed by Patroni and Consul, Redis still uses Sentinel, PgBouncer still reduces database connection pressure, MinIO still externalizes file uploads, and Loki, Grafana, and Netdata still provide observability.

This migration is motivated by the strengths of containers in scenarios where fast deployment, reproducibility, high service density, and horizontal scaling are more important than full machine isolation. Because a container is created from an image and shares the host kernel, it can start in seconds, consume fewer resources than a full VM, and be recreated automatically from a declarative stack definition. These characteristics are especially useful for stateless components such as PHP web replicas, WebSocket workers, reverse proxies, and monitoring agents.

Nevertheless, containers do not replace VMs in every scenario. Virtual machines still provide stronger isolation because each one runs its own kernel and behaves like an independent server. This model is often easier to reason about when services depend on static addresses, low-level networking, multicast protocols, or independent operating-system administration. For that reason, this chapter does not present Docker Swarm as universally superior. Instead, it evaluates which parts of the HA architecture benefit from container-

ization and which parts remain more robust or predictable in the original VM-based design.

3.2 Phase 2 Objectives

The containerization phase was guided by six main objectives:

1. **Replace VMs with containers:** The 11 VirtualBox machines are replaced by Docker services running as containers on a shared host kernel.
2. **Use declarative deployment:** The combination of `Vagrantfile`, Ansible inventory, and multiple roles is replaced by a single Docker Compose.
3. **Preserve service responsibilities:** The same logical tiers from Phase 1 are maintained, including Patroni, Consul, Redis Sentinel, PgBouncer, MinIO, Netdata, Loki, and Grafana.
4. **Use Swarm-native service discovery:** Static VM IP addresses are replaced by Docker DNS names and overlay networking.
5. **Introduce self-healing behavior:** Failed containers should be restarted automatically by the Swarm reconciliation loop whenever the desired state differs from the actual state.
6. **Prepare for auto-scaling:** Stateless services such as the web tier and WebSocket tier should be scalable by changing the number of replicas.

3.3 Architecture Comparison: Virtual Machines and Containers

The Phase 1 architecture uses infrastructure-level separation. Each VM behaves like a small server with its own operating system, network interface, services, logs, and persistent disks. This design is close to a traditional on-premises deployment and provides clear fault boundaries. For example, if one web VM crashes, the other VM can continue serving requests through the load balancer.

The Phase 2 architecture uses process-level isolation. Each application component runs inside a container, and Docker provides namespaces, cgroups, images, networks, volumes, and service definitions. Instead of managing machines directly, the operator manages services. A service declares the image to run, the number of replicas, the exposed ports, the environment variables, the mounted configuration files, and the restart policy.

From a resource perspective, Docker is more efficient because it removes ten duplicated guest kernels and most of the repeated operating-system overhead. The VM deployment reserves ~11 GB of RAM for 11 machines, while the containerized deployment can run the same logical services with a much smaller active memory footprint. Startup time also

Aspect	Phase 1: VM-Based HA	Phase 2: Docker Swarm
Isolation model	Separate operating-system kernel per VM	Shared host kernel with container namespaces and cgroups
Deployment unit	Virtual machine plus provisioned services	Docker service created from an image
Provisioning	Vagrant and Ansible roles	Docker Compose stack file and Dockerfiles
Networking	Static private IP addresses and Keepalived VIP	Overlay network, Docker DNS, and published Swarm ports
Recovery model	Manual VM restart or re-provisioning	Automatic container restart and task reconciliation
Scaling model	Add new VM, update inventory, re-run Ansible, update upstreams	Change service replica count with Swarm
Resource cost	Higher, because every VM reserves memory and disk for a full OS	Lower, because containers share the host OS and image layers

Table 3.1: Comparison between the Phase 1 VM architecture and the Phase 2 Docker architecture.

improves because containers do not boot a full operating system, they start only the target process and its dependencies.

However, this efficiency comes with a trade-off. In the VM design, a failure in the kernel of one VM is isolated from the others. In a single-host deployment, all containers depend on the same host kernel. If the host fails, the entire stack becomes unavailable. Therefore, this container-based approach improves process recovery and operational density, but a single-host container deployment does not provide the same node-level high availability as multiple independent VMs.

3.4 Docker Swarm Fundamentals

Docker Swarm is Docker’s native orchestration mode. It allows a group of Docker hosts to behave as one cluster. The user deploys a stack, and the Swarm manager schedules tasks across available nodes. A task is one running instance of a service, normally represented by one container. If a service is declared with two replicas, Swarm attempts to keep two tasks running at all times.

The most important Swarm concepts for this project are:

1. **Node:** A Docker host participating in the Swarm. Nodes can be managers or workers.
2. **Manager:** The control-plane node responsible for scheduling tasks and maintaining cluster state.

3. **Service:** A declarative definition of a containerized component, such as the web application, PgBouncer, or Redis.
4. **Task:** A concrete running instance of a service.
5. **Replica:** The desired number of tasks for a service.
6. **Stack:** A group of related services, networks, volumes, configs, and secrets deployed together.
7. **Overlay network:** A virtual network that allows services to communicate by name, even when containers are distributed across Swarm nodes.

In the VM-based stack used in Phase 1, service discovery combined static IP addresses with Consul for the database cluster state. For example, the web nodes still had to know the address of the load balancer, the database proxy, Redis, and MinIO, while Patroni used Consul to identify the current PostgreSQL leader. In Phase 2, containers connect using DNS names such as `web`, `nginx`, `db1`, `pgbouncer1`, `redis1`, and `minio`. If Docker recreates a container with a new internal IP address, the service name remains stable. This reduces configuration coupling and makes rescheduling easier.

3.5 Containerized Service Mapping

The Docker version preserves the same logical architecture as the VM deployment, but maps each infrastructure role to one or more container services. The resulting stack includes the following tiers:

1. **Entry point:** An *Nginx* service receives HTTP traffic and routes requests to the web and WebSocket tiers.
2. **Application tier:** The PHP web application runs with multiple replicas, allowing requests to be distributed between identical stateless containers.
3. **Real-time tier:** RatchetPHP WebSocket containers use Redis Pub/Sub so that messages are shared across replicas.
4. **Database tier:** Two PostgreSQL containers are managed by Patroni, while Consul provides distributed consensus for leader election.
5. **Connection pooling tier:** PgBouncer containers reduce the number of direct PostgreSQL connections opened by the PHP application.
6. **Session and cache tier:** Redis containers and Sentinel containers preserve centralized session storage and automated failover.
7. **Object storage tier:** MinIO stores uploaded images outside the web containers, avoiding local file-system state.

8. **Observability tier:** Netdata, Loki, Promtail, and Grafana provide metrics and centralized log exploration.

This mapping demonstrates that containerization does not automatically make a system highly available. The HA behavior still comes from the selected distributed components: Patroni for PostgreSQL, Consul for consensus, Sentinel for Redis, PgBouncer for connection pooling, and MinIO for externalized storage. Docker Swarm improves how these components are packaged, started, discovered, and restarted.

3.6 Containerized Logging with Loki, Promtail, and Grafana

Centralized logging is also preserved in the Docker solution, but the ingestion model changes when compared with the VM-based architecture. In Phase 1, each VM needed its own Promtail instance deployed as a sidecar-like companion service. That Promtail process ran next to the local workload, tailed files such as Nginx, Apache, PostgreSQL, Redis, MinIO, or system logs, attached VM and service labels, and pushed those streams to Loki.

In the Docker architecture, the same idea is implemented with fewer moving parts. Instead of installing one Promtail service per VM, Promtail is deployed as a single containerized log agent with access to the Docker log directory and Docker metadata. All application containers write their standard output and standard error streams through the Docker logging driver, and Docker stores those streams as container log files on the host. Promtail mounts the shared Docker log location, discovers the container log files, enriches them with labels such as container name, service name, and stack name, and pushes the resulting log streams to Loki. Grafana then queries Loki and provides the exploration interface for operators.

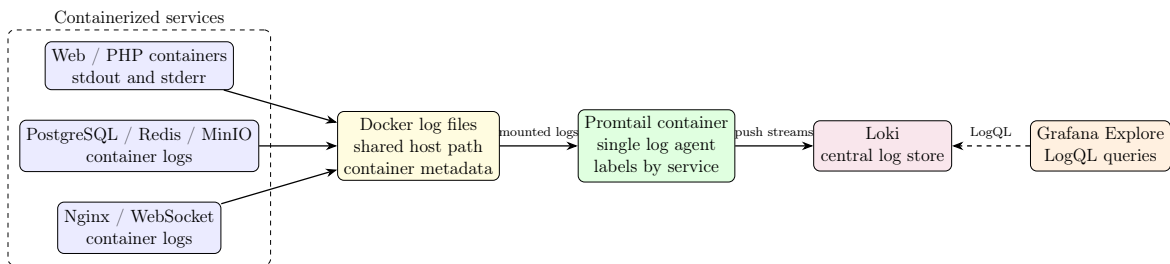


Figure 3.1: Docker logging ingestion with a single Promtail container reading shared Docker logs and pushing them to Loki for Grafana exploration.

This is an important difference between the two deployment models. In the VM solution, Promtail behaves like a sidecar technique replicated on every machine, because logs are physically distributed across independent operating-system instances. In the Docker

solution, the containers share the same host logging substrate, so a single Promtail container can act as the collector for the whole stack. This reduces agent duplication and simplifies deployment, while still preserving the same observability goal: all logs can be searched centrally in Grafana without manually entering each VM or container.

The main trade-off is that the Docker approach depends on the Promtail container having the correct read-only mounts and permissions for the Docker log files. If those mounts are missing, containers may still run correctly but their logs will not be ingested into Loki. For this reason, the Promtail service must be treated as part of the observability tier in the stack definition, alongside Loki and Grafana.

3.7 Networking Changes

Networking is the largest conceptual change between the two phases. In Phase 1, the architecture uses a private VirtualBox network with manually assigned addresses. Keepalived provides a floating virtual IP for the load-balancing tier, and configuration files explicitly reference fixed addresses.

In Phase 2, the stack uses a Docker overlay network. Containers are attached to the same logical network and communicate through Docker's embedded DNS. The operator no longer needs to reserve an address for each service. Instead, service names become the stable interface between components.

The entry point also changes. In Phase 1, Keepalived and VRRP move a virtual IP between two load balancer VMs. In Docker Swarm, published ports and the routing mesh can receive traffic on Swarm nodes and forward it to healthy service tasks. This removes the need for VRRP in the containerized design. The replacement is not identical: VRRP protects a floating address between machines, while the Swarm routing mesh distributes traffic to service tasks. In a multi-node Swarm, this can be powerful; in a single-node Swarm, it is mainly a simplification of local service exposure.

3.8 Self-Healing and Recovery

Docker Swarm continuously compares the desired state declared in the stack file with the actual state of the running containers. If a service is defined with one replica and that container exits, Swarm schedules a replacement task. If a service is defined with two replicas and one fails, Swarm starts another task to return the service to two replicas.

This reconciliation loop is useful for the project because many failures in the VM deployment required manual commands. In the containerized version, a crashed web container can be recreated automatically from its image. A failed WebSocket container can be restarted and rejoin Redis Pub/Sub. A failed database container can be restarted, after which Patroni checks Consul and rejoins the PostgreSQL cluster as either a primary or a replica depending on the current leader state.

Health checks and restart policies are essential for this behavior. A process that is

running is not necessarily healthy. For example, Redis may be started but not responding to authentication, or PostgreSQL may be running but not ready to accept connections. Health checks allow Docker to distinguish between a container that merely exists and a container that is ready to serve traffic.

The containerized failover behavior is therefore split between two layers:

1. **Docker Swarm recovery:** Restarts failed containers and maintains the declared number of replicas.
2. **Application-level HA:** Patroni, Consul, Redis Sentinel, PgBouncer, and MinIO decide which backend is primary, healthy, or available.

This separation is important. Docker can restart a PostgreSQL container, but it does not understand PostgreSQL replication safety by itself. Patroni is still required to avoid unsafe promotion and split-brain scenarios.

3.9 Auto-Scaling Analysis

Docker Swarm supports manual scaling directly through service replica counts. For example, increasing the web service from two replicas to four replicas creates two additional containers. Because PHP sessions are stored in Redis and uploaded files are stored in MinIO, the web tier is stateless enough to support this kind of horizontal scaling.

Automatic scaling is not built into Docker Swarm in the same way that Kubernetes provides a Horizontal Pod Autoscaler. However, auto-scaling can be implemented by connecting metrics collection to Docker service updates. A simple monitor can read CPU and memory usage from the Docker API or from a monitoring stack, compare the values against thresholds, and execute a scaling command when sustained load is detected.

The main rule is that stateless services scale easily, while stateful services require their own clustering mechanisms. Docker can create more containers, but creating more containers is not the same as safely scaling a database or storage system.

3.10 Performance and Failover Observations

The measured Docker deployment showed that static HTTP requests performed almost identically to the VM version, with only a very small latency increase. This indicates that containerization itself does not significantly penalize simple web traffic. The database-backed endpoint showed higher latency in Docker, mainly because the request path included additional bridge-network hops and an Nginx stream proxy for TCP routing. In future iterations, replacing the Nginx stream proxy with a dedicated HAProxy container would likely reduce this overhead.

The Docker version still achieved successful operation under load and preserved the main HA behavior. During PostgreSQL failover testing, almost all requests succeeded, with a small interruption during the leader promotion window. The main advantage of

Service	Auto-scale suitability	Reason
Web application	High	The service is stateless after sessions and uploads are externalized to Redis and MinIO.
WebSocket service	High, with care	Redis Pub/Sub synchronizes messages, but connection stickiness may be needed for long-lived clients.
Nginx	Medium	It can be replicated in a multi-node Swarm, but published ports and routing rules must be planned carefully.
PgBouncer	Medium	Additional replicas may help distribute client connections, but the service already exists to pool database traffic.
PostgreSQL	Low	It is stateful and controlled by Patroni; scaling requires replication design, not simple replica duplication.
Redis	Low	Redis topology is controlled by master-replica replication and Sentinel quorum.
MinIO	Low in this deployment	Safe scaling requires a planned distributed MinIO layout and persistent volumes.

Table 3.2: Suitability of each tier for Docker Swarm auto-scaling.

Docker in this scenario was recovery speed: restarting a container and allowing it to rejoin the cluster is faster than booting a full VM and then starting its services.

These observations show a realistic trade-off. Docker improves density, deployment speed, and recovery time, but the extra container networking layers can add latency for database paths. The VM version remains more predictable for low-level network routing, while the Docker version is more flexible for application-tier scaling and rapid redeployment.

3.11 Advantages and Disadvantages

The VM-based architecture has the following advantages:

1. **Stronger isolation:** Each VM has its own kernel and operating-system boundary.
2. **Clear infrastructure model:** Static IPs and standard Linux tools make the system easy to inspect with SSH, system logs, and service managers.
3. **Better node-level resilience:** Failure of one VM does not necessarily imply failure of all services.
4. **Natural support for VRRP:** Keepalived and multicast-based failover are easier to use in a VM network.

Its disadvantages are also significant:

1. **High resource consumption:** Each VM reserves memory, disk, and CPU capacity for a full operating system.

2. **Slow provisioning:** Booting and configuring multiple machines takes longer than starting containers.
3. **Configuration sprawl:** Static IPs and service addresses appear in many files.
4. **Manual recovery:** Failed machines often require explicit restart or re-provisioning.

The Docker Swarm architecture has the following advantages:

1. **Lightweight execution:** Containers share the host kernel and reuse image layers.
2. **Fast startup and redeployment:** Services can be recreated in seconds from images.
3. **Declarative desired state:** The stack file describes services, networks, volumes, and replicas in one place.
4. **Built-in service discovery:** Containers use DNS names instead of hardcoded IP addresses.
5. **Self-healing:** Swarm restarts failed tasks to match the declared replica count.
6. **Simple scaling for stateless tiers:** Web and WebSocket replicas can be increased without adding new VMs.

Its disadvantages are the following:

1. **Weaker isolation:** All containers depend on the same host kernel.
2. **Single-host SPOF in this lab:** If the Docker host fails, the whole stack fails.
3. **Networking overhead:** Bridge and overlay networking can add latency, especially in database paths.
4. **Stateful-service complexity:** Databases, Redis, and MinIO require careful volume and replication design.
5. **Different operational model:** Troubleshooting requires Docker, image, network, and container knowledge in addition to Linux administration.

This page intentionally left blank.

Chapter 4

Cloud Deployment Alternative

4.1 Introduction

This part of the project was not implemented in the final prototype, but it was considered during the planning phase. The main reason for analyzing a cloud alternative was that the teacher offered a 50 USD voucher for Google Cloud Platform (GCP). Because of that, the cloud version was mainly designed around Google services and around the way those services could replace the virtual-machine-based architecture used in the laboratory environment.

Although the proposal below focuses on GCP, the same architectural logic is not exclusive to Google Cloud. One important lesson from the semester is that most cloud providers offer equivalent services for the same infrastructure problems. A load balancer, a managed database, object storage, container execution, monitoring, and centralized logging exist in different forms across GCP, AWS, and Azure. The names and configuration details change, but the design goals are usually similar: reduce manual server administration, improve scalability, externalize state, and make the system easier to operate.

Therefore, this chapter should be understood as a cloud-oriented alternative rather than a finished implementation. It explains how the same application could be redesigned using managed cloud services, with a stronger focus on containers and service abstraction instead of manually provisioning several Vagrant virtual machines.

4.2 Proposed GCP-Based Architecture

The VM-based prototype showed that the application can be improved by separating the load-balancing tier, web tier, database, sessions, object storage, WebSocket coordination, monitoring, and logging. In a cloud deployment, the same separation could be achieved with managed services instead of manually maintaining each VM.

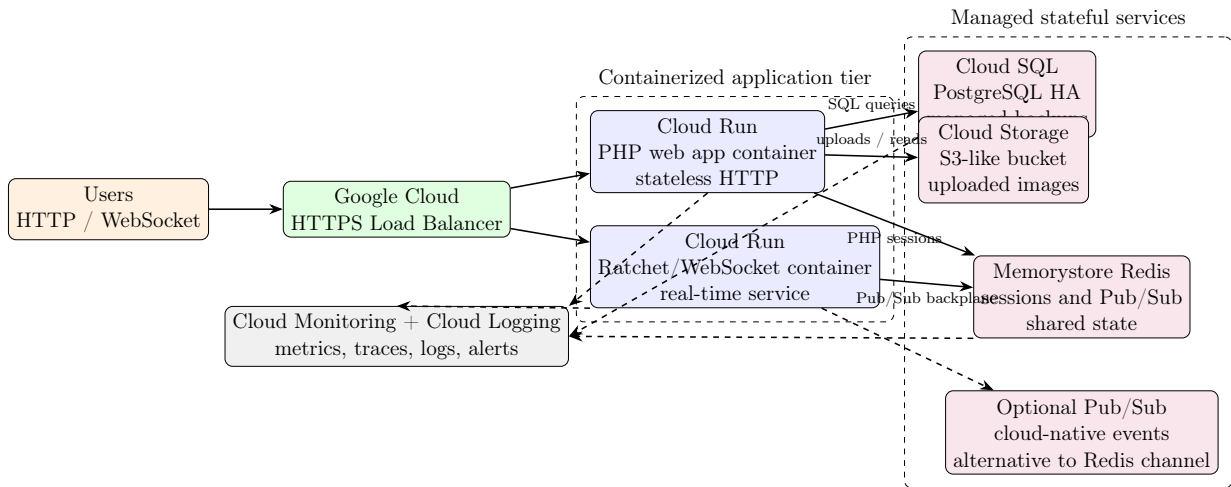


Figure 4.1: Proposed GCP cloud alternative using Cloud Run containers and managed stateful services.

A possible GCP solution would use *Cloud Run* as the main container execution platform for the PHP web application. Cloud Run is appropriate for stateless HTTP services because it runs containers, scales them automatically according to demand, and removes the need to manage the underlying servers. The PHP application could be packaged into a container image and deployed as one or more Cloud Run services behind a Google Cloud HTTPS Load Balancer. This would replace the manually configured Nginx and Keepalived load-balancing layer used in the VM prototype.

For the database layer, *Cloud SQL for PostgreSQL* would replace the manually configured PostgreSQL, Patroni, Consul, HAProxy, and PgBouncer stack. Cloud SQL provides a managed PostgreSQL service with automated backups, maintenance operations, monitoring integration, and high-availability options. This would significantly reduce operational complexity because failover and database lifecycle management would be handled by the provider rather than by custom VM roles.

For uploaded images, *Cloud Storage* would replace local disks and MinIO. The application would store uploaded files in a bucket instead of writing them to the web-server file system. This follows the same principle used in the final VM architecture: web containers should remain stateless, while user-generated files are stored in a dedicated object-storage service. In AWS this role would commonly be handled by S3, and in Azure by Blob Storage.

For sessions and fast shared state, *Memystore for Redis* could replace the Redis Sentinel cluster. Instead of maintaining Redis replicas and Sentinel processes on storage VMs, the application would connect to a managed Redis instance. This would provide a cleaner solution for PHP sessions and for WebSocket Pub/Sub coordination. Depending on the final design, Redis could continue to act as the shared communication backplane between WebSocket instances.

For real-time communication, there would be two possible directions. The first option would be to keep RatchetPHP in a containerized service and use Redis Pub/Sub through

Memorystore, similar to the VM prototype. The second option would be to redesign the real-time layer using a cloud-native messaging service such as *Pub/Sub*. The first option would require fewer code changes, while the second would align better with managed cloud patterns.

4.3 Container-Oriented Deployment Model

The most important difference between this cloud alternative and the implemented laboratory prototype is the shift from virtual machines to containers. Instead of creating many VMs with fixed IP addresses, each application component would be packaged as a container image and deployed through a managed runtime.

A simplified container-based deployment could follow this structure:

- **Web application container:** Contains Apache or PHP-FPM, the PHP application code, and the configuration needed to connect to Cloud SQL, Cloud Storage, and Redis.
- **WebSocket container:** Runs the RatchetPHP server and connects to Redis or Pub/Sub for cross-instance message synchronization.
- **Managed PostgreSQL:** Provided by Cloud SQL, removing the need to operate Patroni, Consul, and database VMs manually.
- **Managed object storage:** Provided by Cloud Storage, replacing local upload directories and MinIO.
- **Managed Redis:** Provided by Memorystore, replacing the self-managed Redis Sentinel cluster.
- **Monitoring and logging:** Provided by Google Cloud Monitoring and Cloud Logging, replacing or complementing Netdata, Loki, and Grafana.

This approach would make deployment faster and more repeatable. Instead of provisioning each VM and configuring services through Ansible roles, the group would build container images, define environment variables, configure service accounts, and deploy the containers to Cloud Run or a similar container platform. The same concept could also be implemented with AWS App Runner or ECS, or with Azure Container Apps.

4.4 Expected Benefits and Trade-Offs

The main benefit of the cloud alternative would be operational simplicity. Many components that required manual configuration in the VM prototype would become managed services. Database failover, object storage durability, Redis availability, centralized logs,

metrics, and load balancing would be handled mostly by the provider. This would allow the team to focus more on the application and less on maintaining infrastructure.

Another benefit would be elasticity. In the VM-based environment, the number of machines is fixed and limited by the host computer resources. In a cloud environment, services such as Cloud Run can scale horizontally based on traffic, and managed storage services can grow without manually adding disks or VMs. This is one of the reasons why the industry increasingly prefers containerized and managed platforms: they make scaling and recovery easier to automate.

However, this alternative also has trade-offs. Cloud services introduce cost management concerns, provider-specific configuration, identity and access management complexity, and possible vendor lock-in. The 50 USD voucher would be useful for experimentation, but a real production deployment would need budget control, security hardening, backup policies, and careful monitoring of resource usage.

Overall, the cloud version would preserve the same architectural principles developed in the VM prototype, but would implement them with managed services and containers. The project therefore helped clarify why both approaches are useful: VMs are excellent for learning how each service works internally, while cloud containers and managed services are often better for reducing operational effort in real-world deployments.

Chapter 5

Conclusion

This project made it possible to understand, in a practical way, how a simple monolithic application can be progressively transformed into a more available and scalable architecture. Even though the final solution was not extremely complex from an application-development point of view, the work showed that most of the difficulty in high availability is not only in the code itself, but in the way services are separated, deployed, monitored, and connected.

A large part of the project ended up being focused on virtual machines and infrastructure configuration. This was useful because it exposed the operational cost of managing a multi-node system: IP addressing, provisioning, service dependencies, failover behavior, load balancing, storage, sessions, and monitoring all had to be configured explicitly. By starting from a single VM and then introducing load balancers, database high availability, Redis Sentinel, WebSocket synchronization, MinIO, Netdata, Loki, and Grafana, the group could see how each additional layer removes one limitation but also introduces new complexity.

At the same time, the project also helped clarify the difference between virtual-machine-based deployment and containerization. Virtual machines provide strong isolation and are easy to understand conceptually because each service can be treated as a separate server. However, they are heavier, slower to recreate, and more demanding in terms of local resources. Containers, by contrast, are lighter, faster to start, easier to replicate, and better aligned with automated deployment and orchestration. This explains why the industry has increasingly moved toward container-based platforms: they reduce operational friction and make horizontal scaling more practical, especially when combined with tools such as Docker and Kubernetes.

The main limitation of this work was that the architecture was implemented in a constrained laboratory environment. Some components were simplified, and the focus was stronger on Vagrant-based VMs than on a full containerized or cloud-native deployment. Nevertheless, this limitation was also valuable, because it made the trade-offs visible. Managing many VMs manually or semi-automatically quickly becomes complex, which reinforces the motivation for Infrastructure-as-Code, containers, orchestration, and centralized observability.

Overall, the project achieved its main objective: it demonstrated why stateful com-

ponents such as databases, sessions, uploads, and WebSocket coordination must be externalized or carefully managed before an application can scale horizontally. It also showed that high availability is not a single technology, but a combination of redundancy, failover, shared state, monitoring, and operational discipline. As future work, the same architecture could be reimplemented with containers and orchestration to compare resource usage, deployment speed, maintainability, and scalability against the VM-based approach used in this prototype.

Bibliography

- [1] IBM. “High availability overview,” Accessed: Jun. 10, 2026. [Online]. Available: <https://www.ibm.com/docs/en/i/7.5.0?topic=availability-high-overview>
- [2] YouTube. “Stateful and stateless layers explained,” Accessed: Jun. 24, 2026. [Online]. Available: https://www.youtube.com/watch?v=20tpk8A_xa0
- [3] Grafana Labs. “Install Loki,” Accessed: Jun. 30, 2026. [Online]. Available: <https://grafana.com/docs/loki/latest/setup/install/>

This page intentionally left blank.

Appendix A

Report given initially

Final Project – Cloud Computing and Virtualization v2

MEI-IoT – Masters in Informatics Engineering: Internet of Things

Prof. Renato Panda (renato.panda@ipt.pt)

2026-05-13

Table of contents

1	Final Project – Cloud Computing and Virtualization	2
1.1	Introduction	2
1.2	Learning Outcomes	2
1.3	Objectives	2
1.4	Project Context	2
1.5	Group Work	3
1.6	Project Tasks	3
1.6.1	A. Virtual Machines (On-Premises IaaS Simulation)	3
1.6.2	B. Containers & Cloud-Native (On-Premises or Public Cloud)	4
1.7	Intermediate Checkpoints and Technical Discussions	4
1.7.1	Checkpoint 1 — Original System Analysis and Initial Plan	4
1.7.2	Checkpoint 2 — Solution A Architecture and Prototype	5
1.7.3	Checkpoint 3 — Solution A Consolidation and Solution B Plan	5
1.7.4	Checkpoint 4 — Solution B Prototype and Comparative Analysis	5
1.7.5	Use of AI and External Assistance	6
1.7.6	Decision Log	6
1.8	Deliverables	6
1.9	Submission Guidelines	7
1.10	Suggestions and Tips	7
1.11	Resources	8
1.12	Assessment Criteria	8
1.13	Deadline	9

1 Final Project – Cloud Computing and Virtualization

1.1 Introduction

The project is designed to consolidate your understanding of modern cloud and virtualization technologies, focusing on the design, deployment, and critical analysis of scalable and resilient infrastructures and applications.

1.2 Learning Outcomes

Upon successful completion, you will be able to:

- Analyze and critique application architectures for scalability and resilience.
- Design and implement distributed systems using VMs and containers.
- Apply cloud-native and automation tools to real-world scenarios.
- Justify architectural decisions and reflect on trade-offs.

1.3 Objectives

By completing this project, you will:

- Develop practical skills in:
 - Provisioning tools
 - Load balancing
 - High availability
 - Service discovery
 - Containers and cloud-native technologies
 - Scalability and elasticity
- Learn to critically analyze and improve real-world application architectures.
- Demonstrate the ability to justify design decisions and reflect on trade-offs.

1.4 Project Context

Modern infrastructures must be resilient and capable of horizontal scaling to fully leverage virtualization and cloud computing. In this project, you will start with **Project O** — a simple PHP web application provided as a baseline Vagrant setup — and reimagine its deployment so that it becomes **scalable, resilient, and highly available**.

You will explore two distinct approaches (A and B), each emphasizing different aspects of cloud, containerization, and virtualization.

Project O (the original application) includes:

- Static content
- Data loading from external APIs (e.g., cat pictures)
- JavaScript libraries
- Sessions
- File uploads
- Relational database
- WebSocket communication

1.5 Group Work

- **Default:** The project should be done in groups of 2 students, but both students must actively participate to ensure hands-on experience with all tools and concepts.
- **Exception:** Individual work is allowed if justified (e.g., scheduling conflicts, other commitments). Please contact the instructor in advance to discuss and approve individual work.

1.6 Project Tasks

You will design, implement, and analyze two alternative architectures for the provided web application:

1.6.1 A. Virtual Machines (On-Premises IaaS Simulation)

- Use **Vagrant** and virtual machines to deploy the application in a more robust, distributed, and scalable manner, simulating an on-premises IaaS environment.
- Use **Ubuntu** or **Alpine Linux** (recommended to save resources) as the VM operating system.
- You are free to explore and combine any of the tools and techniques covered in class, including (but not limited to):
 - **Automation:** Ansible (provisioning, configuration management)
 - **Load balancing:** nginx (or similar)
 - **High availability:** Keepalived (VRRP failover for load balancers)
 - **Database HA:** repmgr (PostgreSQL streaming replication), Patroni (automatic PostgreSQL failover)
 - **Sessions:** centralized session storage (e.g., Redis, Memcached)
 - **Storage:** shared/distributed storage for file uploads (e.g., GlusterFS, NFS)
 - **Service discovery:** Consul (automatic registration and load balancer updates)
 - **Monitoring & Observability:** Prometheus, Grafana, Loki, SigNoz

1.6.2 B. Containers & Cloud-Native (On-Premises or Public Cloud)

- Use **Docker containers** and orchestration (Docker Swarm, Kubernetes, or similar) to deploy the application, either on-premises or using a public cloud provider.
- Students will have access to **Google Cloud Platform (GCP) education credits** to explore managed cloud services (Compute Engine, Cloud SQL, Cloud Storage, GKE, Cloud Load Balancing, etc.).
- You may also explore **GCP emulators** locally before deploying to the cloud:
 - [gcloud emulators](#)
 - [gcloud beta emulators](#)
 - [fake-gcs-server](#) (GCS emulator)
 - [fullstorydev/emulators](#) (Pub/Sub, Bigtable, etc.)
- Integrate IaaS or cloud services as appropriate for your design. The same topics listed for Option A are valid here, applied in a container/cloud context.

For both approaches, you are expected to:

- Critically analyze **Project O** and its original architecture.
- Propose and implement an improved, scalable, resilient, and highly available architecture.
- Justify your design choices and discuss any limitations or trade-offs.

1.7 Intermediate Checkpoints and Technical Discussions

During the project, you must complete several intermediate checkpoints. Each checkpoint requires a short Moodle submission, preferably **3–4 slides**, to support a brief **10-minute technical discussion** with the practical class instructor.

The goal is not to create polished mini-presentations. The checkpoints are intended to show progress, justify decisions, identify difficulties, and demonstrate that you understand the system you are building. The exact timing of each discussion may be adjusted or agreed with the practical class instructor.

1.7.1 Checkpoint 1 — Original System Analysis and Initial Plan

Suggested date: 21/05/2026

Present:

- A simple architecture diagram of Project O.
- Main limitations of the original application regarding scalability, resilience, high availability, sessions, file uploads, database, and WebSockets.
- Initial requirements for improving the system.
- Initial plan for Solution A using Vagrant/VMs.

1.7.2 Checkpoint 2 — Solution A Architecture and Prototype**Suggested date:** 04/06/2026

Present:

- Planned architecture for Solution A.
- Current implementation status.
- Tools selected and why they were selected.
- Evidence of progress, such as commands, screenshots, configuration snippets, logs, or a short demo.
- Main technical risks and next steps.

1.7.3 Checkpoint 3 — Solution A Consolidation and Solution B Plan**Suggested date:** 18/06/2026

Present:

- Implemented architecture for Solution A.
- Differences between planned and implemented architecture.
- Evidence of testing, such as load balancing, failure simulation, persistence, monitoring, or performance tests.
- Limitations of Solution A.
- Initial architecture and rationale for Solution B using containers, orchestration, cloud services, or a hybrid approach.

1.7.4 Checkpoint 4 — Solution B Prototype and Comparative Analysis**Suggested date:** 25/06/2026

Present:

- Implemented or nearly implemented architecture for Solution B.
- Current technical status and remaining work.
- Comparison between Project O, Solution A, and Solution B.
- Final plan for the report, demo, and defense.

1.7.5 Use of AI and External Assistance

You may use AI-based tools and external resources for learning, debugging, documentation, and exploration. However, you remain fully responsible for understanding, validating, and explaining all submitted work.

The final report should include a short note describing which AI tools or external assistance were used, for what purpose, and how the outputs were validated.

1.7.6 Decision Log

The final report should include a brief decision log summarizing relevant architectural and technical decisions. For each major decision, briefly mention:

- The problem or requirement.
- Alternatives considered.
- Chosen solution.
- Reason for the choice.
- Limitations or trade-offs.

Examples of useful decisions to document:

- Why use Redis, Memcached, the SQL database or XPTO for sessions?
- Why use NFS, GlusterFS, or object storage for uploads?
- Why use nginx, HAProxy, or a cloud load balancer?
- Why use Docker Compose, Docker Swarm, Kubernetes, GCP, or a hybrid approach?
- Why implement or not implement database high availability?

1.8 Deliverables

Submit a **ZIP file** containing all relevant materials:

- **Project Report** (recommended structure):
 1. **Introduction and Objectives**
 2. **Analysis of the Original Application**
 - System architecture diagram
 - Technologies used
 - Identified issues and limitations
 - (Optional) Performance metrics (e.g., using **hey** or **Vegeta**)
 3. **Proposed Solutions**
 - Separate sections for A (VMs) and B (Containers/Cloud)
 - Planned/ideal vs. implemented architecture (with architecture diagrams)

- Main components, rationale, and implementation details
- (Optional) Performance metrics
- 4. **Engineering Process and Decision Log**
 - Main decisions, alternatives considered, and trade-offs
 - Evidence of testing, debugging, and iteration
 - Short note on the use of AI tools or external assistance
- 5. **Comparison and Conclusions**
 - Comparison between Project O, Solution A, and Solution B
 - Key takeaways and lessons learned
- **Configuration/code and instructions** to replicate/run your solution.
- **Slides and demo** (max 15 minutes; slides may be submitted after the report).

1.9 Submission Guidelines

- Submit your ZIP file via the platform or as instructed by the professor.
- Ensure all files are clearly named and organized.
- Include a README with student name(s), instructions, and any special notes.

1.10 Suggestions and Tips

- **Critical Thinking:**

Discuss features you considered, even if not implemented. Explain your reasoning and trade-offs.
- **Potential Features to Explore:**
 - Three-layer architecture (load balancer, web, database)
 - Serving the website correctly (partially or fully)
 - Manual or semi-automatic scaling of instances
 - Automatic service discovery and dynamic load balancer updates (e.g., Consul)
 - Auto-scaling the web layer (e.g., based on schedule or CPU load)
 - High availability for the load balancer (e.g., Keepalived / VRRP, cloud LB)
 - High availability for the database (e.g., repmgr, Patroni, managed cloud DB)
 - Centralized session storage to support horizontal web scaling (e.g., Redis)
 - Shared/distributed storage for file uploads (e.g., GlusterFS, NFS, cloud object storage)
 - Monitoring & observability stack (e.g., Prometheus + Grafana, Loki, SigNoz)
 - Automated deployment and configuration management (e.g., Ansible, Terraform/OpenTofu)
- **Documentation:**

Use diagrams, tables, and clear explanations. Reference all external resources and tools used.

1.11 Resources

- **Project O repository:** https://github.com/renatopanda/ipt_cloud_course_project *(link may be updated — check the course platform for the latest version)*
- **Recommended tools:** Vagrant, Docker, Google Cloud Platform, and tools explored during classes: Ansible, nginx, Consul, Keepalived, repmgr, Patroni, Redis, GlusterFS, Prometheus, Grafana, Loki, SigNoz, Docker Swarm, Kubernetes, Terraform/OpenTofu, hey, Vegeta, etc.
- **GCP Emulators (Option B — local development):**
 - [gcloud emulators](#)
 - [gcloud beta emulators](#)
 - [fake-gcs-server](#) (GCS emulator)
 - [fullstorydev/emulators](#) (Pub/Sub, Bigtable, etc.)

1.12 Assessment Criteria

Your project will be evaluated based on the following **objective criteria**:

- **Analysis and Planning (15%)**
 - Depth and clarity of the original system analysis
 - Identification of relevant issues and requirements
- **Design and Implementation (35%)**
 - Appropriateness and justification of architectural choices
 - Correctness and completeness of the implemented solutions (A & B)
 - Use of relevant tools and technologies
- **Documentation and Communication (15%)**
 - Quality, clarity, and organization of the report and diagrams
 - Clear, reproducible instructions and code/configuration
- **Critical Reflection (15%)**
 - Discussion of limitations, trade-offs, and lessons learned
 - Comparison between original and proposed solutions
- **Intermediate Checkpoints and Technical Discussion (10%)**
 - Evidence of steady progress and response to feedback
 - Ability to explain decisions, risks, and implementation status
- **Final Presentation and Defense (10%)**
 - Clarity and effectiveness of the oral presentation/demo

- Ability to answer questions and demonstrate understanding

All students must attend the project defense and demonstrate a clear understanding of the solution.

1.13 Deadline

- **Final submission:** around **01/07/2026**, to be confirmed on the course platform.
- Intermediate checkpoint dates are suggested dates and may be adjusted by the practical class instructor.

For reference and updates, please consult the project repository and the course platform.

Appendix B

Attachments

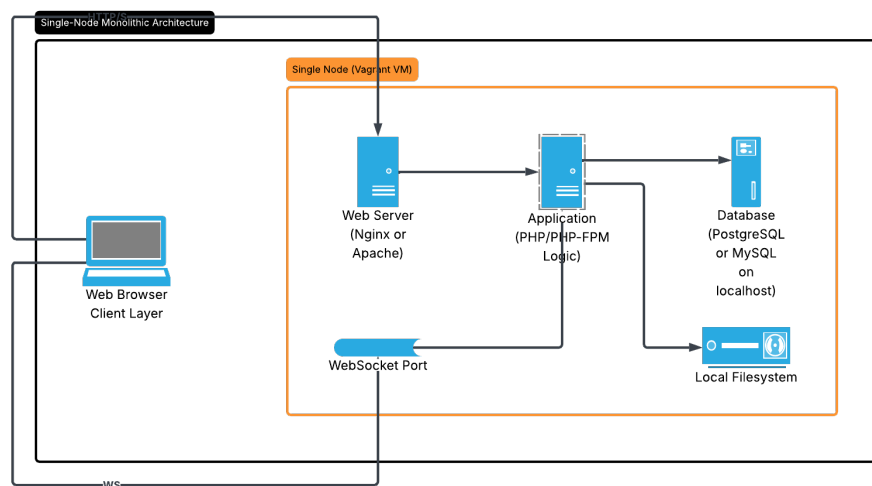


Figure B.1: Architecture of the base project.

```
vagrant@webapp:~$ ls -lah /vagrant/app/public_html/gallery/
total 212K
drwxrwxrwx 1 vagrant vagrant  0 Jun  4 2026 .
drwxrwxrwx 1 vagrant vagrant 4.0K May 20 16:36 ...
-rwxrwxrwx 1 vagrant vagrant 118K May 20 16:36 capa_Inicho.jpg
-rwxrwxrwx 1 vagrant vagrant  88K Jun  4 2026 vegeta-plot.jpg
```

Figure B.2: Initial stack image upload stored in the local gallery directory.

B.1 Local Gallery Directory Inspection

```
vagrant@webapp:~$ ls -lah /vagrant/app/public_html/gallery/
```

```
total 212K
drwxrwxrwx 1 vagrant vagrant    0 Jun  4  2026 .
drwxrwxrwx 1 vagrant vagrant 4.0K May 20 16:36 ..
-rwxrwxrwx 1 vagrant vagrant 118K May 20 16:36 capa_Tnicho.jpg
-rwxrwxrwx 1 vagrant vagrant  88K Jun  4  2026 vegeta-plot.jpg
```

B.2 Local PHP Session Directory Inspection

```
vagrant@webapp:~$ sudo ls -lah /var/lib/php/sessions
total 12K
drwx-wx-wt 2 root      root      4.0K Jun  4 21:42 .
drwxr-xr-x 4 root      root      4.0K Jun  4 10:14 ..
-rw----- 1 www-data www-data  31 Jun  4 21:43 sess_rhucf77ap93u5a6vbfrlv6urr0
```

B.3 Network and Process Inspection

```
vagrant@webapp:~$ sudo ss -tulpn | grep 8000
vagrant@webapp:~$ sudo ss -tulpn | grep 5432
tcp    LISTEN 0      244          127.0.0.1:5432      0.0.0.0:*
↪  users:(("postgres",pid=15436,fd=5))
```

B.4 Vegeta Load Test and Plot Generation

```
echo "GET http://192.168.44.10/db.php" | vegeta attack -duration=60s -rate=200 |
↪ tee resultados.bin | vegeta report
vegeta plot resultados.bin > grafico.html
```

B.5 Vegeta Performance Results

```
Requests      [total, rate, throughput]    12000, 200.02, 7.90
Duration      [total, attack, wait]        1m30s, 59.995s, 30.001s
Latencies     [min, mean, 50, 90, 95, 99, max] 146.137ms, 28.908s, 30.001s,
↪ 30.001s, 30.001s, 30.001s, 30.021s
Success       [ratio]                      5.92%
Status Codes  [code:count]                 0:11289 200:711
Error Set:
```

```
Get "http://192.168.44.10/db.php": context deadline exceeded (Client.Timeout
↪ exceeded while awaiting headers)
Get "http://192.168.44.10/db.php": dial tcp 0.0.0.0:0->192.168.44.10:80: i/o
↪ timeout (Client.Timeout exceeded while awaiting headers)
```

B.6 Apache Worker Exhaustion Log

```
[Thu Jun 04 21:36:33.417491 2026] [mpm_prefork:error] [pid 13373] AH00161: server
↳ reached MaxRequestWorkers setting, consider raising the MaxRequestWorkers
↳ setting
```

B.7 Resource Monitoring and Log Auditing

```
htop
cat /var/log/apache2/error.log
```

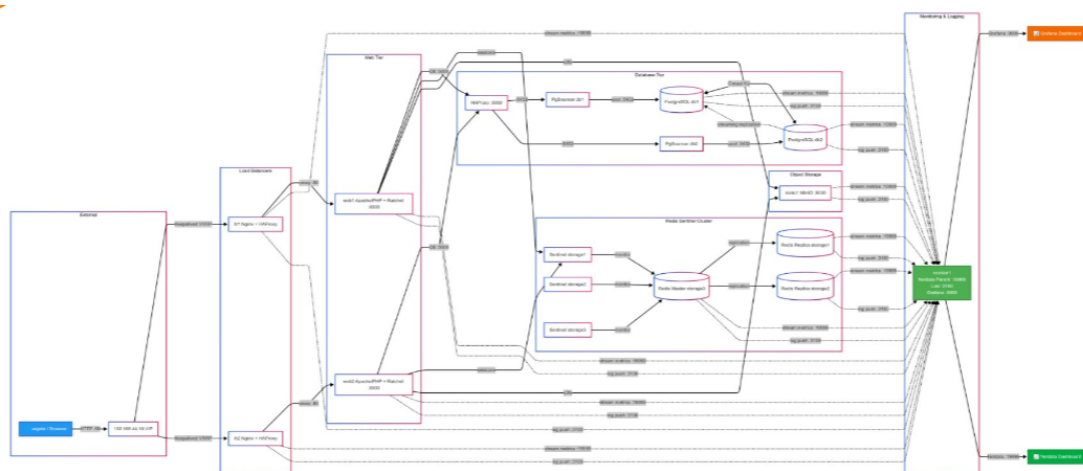


Figure B.3: Proposed high-availability architecture.



Figure B.4: Successful Jelbrek execution.